

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf\_f6b96576-946\agent-a24a3598fe06fc2ee.jsonl`
- SHA-256: `f76efdc2c265484d48766e3a0f9f35fd8da1fe7d50975eafc2010463f36b12a1`
- Source modified: `2026-07-06T18:03:54+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf\_f6b96576-946`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T18:00:43.632Z)

You are a COMPLETENESS & REGRESSION reviewer.

REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer

FULL DIFF UNDER REVIEW (read first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-

indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff

BACKGROUND: wf294 = the Cloud Run rally-worker recorded status="success" / 0 segments even when the WASB detector TIMED OUT and produced no trajectory ? a detector failure indistinguishable from

a legit 0-rally video. An earlier change (already reviewed CLEAN) fixed the two core detector paths

(healthcheck-not-ready, no-trajectory-abort) to return backend.results.Failure, and made worker.cmd_infer's _fail() write+push a status="failed" report.json + done-marker.

THIS REVIEW covers the FOLLOW-UP completeness work that closed the same silent-failure CLASS on adjacent paths (a prior review confirmed these were the last gaps). Three sub-changes:

F1 ? backend/pipeline/segmenters/trajectory_hybrid.py process_video: the pre-detection config/env/

input abort paths now return Failure instead of bare []: unknown sport (KeyError), missing API

key (skip_ai=False), unknown provider (KeyError), invalid/<=0 video duration, and _resolve_runner

NotImplementedError (detector_impl=native on a family with no native runner). A genuine empty match (trajectory produced but zero candidate windows) still returns bare [].

F2 ? backend/pipeline/segmenters/yolo_hybrid.py (the model-default segmenter): same treatment ? _resolve_provider now returns Failure (not None) on bad sport / missing key / unknown provider;

process_video returns Failure on invalid duration; return annotation widened; imports Failure.

F3 ? the AI-handoff "all windows errored" path in BOTH trajectory_hybrid.py (inline Phase 3) and yolo_hybrid.py (_run_ai_handoff): the per-window handoff fn now returns (valid_segments, errored)

where errored=True iff the window yielded NO verdict (clip extraction failed OR provider metrics['error']). If candidate windows > 0 AND every handed-off window errored, process_video

returns a Failure(is_recoverable=True). If ANY window ran (even returning no rallies), it stays a

Success (bare list). skip_ai (fully-local) has no provider and never reaches this path.

CONTRACT: every caller (worker.cmd_infer:326, main.py:1557, orchestration.py:806) branches on isinstance(result, Failure). worker._fail writes status="failed" report.json + marker.

trajectory_hybrid is NOT mypy-quarantined (must stay mypy-clean); yolo_hybrid + worker.__main__ ARE
quarantined (mypy.ini ignore_errors). CI: ruff 0.15.16, mypy 2.1.0 (mypy backend training),
run_all_tests.py, coverage floor 70%. Current state: ruff clean, mypy clean, 1740 passed / 3
skipped, coverage 85.85%. So do NOT speculate that "tests might fail" ? cite concrete code
defects.

Report ONLY real defects with exact file:line: a wrong/edge behavior (off-by-one, an all-errored
run that stays "success", or a partial/legit-empty run wrongly failed), a broken contract, a
thread-safety/ordering bug, a regression vs the prior behavior, a convention violation, or a
test
that asserts the wrong thing / has a gap. If you cannot ground a concern in the actual code, DO
NOT
report it.

YOUR LENS:

- F1/F2: did converting the config/env/input paths to Failure MISS any sibling bare-[] path that
is
also a real failure, or WRONGLY convert a legit-empty path? Enumerate every remaining `return
[]`
/ `return gemini\_segments` / `return segments\_data` in both process_video methods and classify
each as correct-empty vs should-be-failure.
- Does F2's yolo change interact correctly with the caller? _run_ai_handoff can now return a
Failure
? confirm process_video checks isinstance(handoff, Failure) BEFORE passing it to _populate_db
(which would iterate a Failure). Confirm _resolve_provider's None->Failure change has no
remaining
`is None` check anywhere.
- fusion_hybrid inherits trajectory_hybrid.process_video ? does F1/F3 apply correctly to fusion,
and
do the fusion tests still hold? Any OTHER caller/test in the repo that assumed these paths
return
[] (grep for process_video result asserts, unwrap_or_failure, len() on the result)?
- Control plane: a Failure -> worker _fail -> status="failed" report.json. Confirm no consumer
(orchestration.cached_process_result, job_worker._resume_one_remote_job, compute.cloud_run)
regresses on the newly-failed config/handoff cases.

Report concrete completeness gaps or regressions with file:line + a scenario.

2. assistant (2026-07-06T18:00:46.163Z)

I'll start by reading the full diff under review, then examine the relevant source files to
verify the completeness and correctness of the changes.

3. user (2026-07-06T18:00:48.239Z)

```
1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..7987278 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
```

```

20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,14 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Result contract (the ABC declares Success|Failure). Anything that means the
run could
34     +     # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
35     +     # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
36     +     # not ready, or run_predict timing out / aborting with no trajectory ? returns
a ``Failure``
37     +     # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
38     +     # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
39     +     # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
40         label = self.display_label
41         # --- Sport profile + AI provider wiring (identical across segmenters) ---
42         sport_name = self.config.sport
43     @@ -185,7 +192,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
44         sport_profile = sport_registry.get(sport_name)
45         except KeyError as e:
46             logger.error(str(e))
47     -         return []
48     +         # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
49     +         return Failure(str(e))
50
51         idx_cfg = self.config.indexing
52         skip_ai = bool(idx_cfg.skip_ai_handoff)
53     @@ -205,24 +213,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
54         if not api_key:
55             from backend.pipeline.segmenters._keyhint import api_key_hint
56
57     -         logger.error(
58     +         msg = (
59             f"{api_key_env_var} environment variable is not set! "
60             f"To run the {provider_name} handoff, set your API key. "
61             + api_key_hint(api_key_env_var)
62         )
63     -         return []
64     +         logger.error(msg)
65     +         # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
66     +         # marked failed instead of a silent empty "success" (mirrors gemini;
the worker's
67     +         # own 0-segment diagnostic names this missing-key case as a top cause).
68     +         return Failure(msg, is_recoverable=True)
69         try:
70             provider = provider_registry.get(
71                 provider_name, api_key=api_key, model_name=model_name
72             )
73         except KeyError as e:
74             logger.error(str(e))
75     -         return []

```

```

76      +                return Failure(str(e))
77
78      video_duration = get_video_duration(video_path)
79      if video_duration <= 0:
80          logger.error("Invalid video duration.")
81      -        return []
82      +        # An unreadable/broken video, not a 0-rally match.
83      +        return Failure(
84      +            message=f"{label}: invalid/unreadable video duration
({video_duration}).",
85      +        )
86      fps, frame_width = self._probe_fps_width(video_path)
87
88      # --- Runner config + windowing thresholds ---
89      @@ -233,9 +248,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
90          runner, runner_params = self._resolve_runner(idx_cfg)
91      except NotImplementedError as e:
92          # e.g. detector_impl="native" for a family without a native runner
93      (tracknet, or
94      -        # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
95      +        # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
96      +        # 0-rally match ? refuse cleanly as a Failure instead of crashing or a
false success.
97      logger.error("%s: %s", label, e)
98      -        return []
99      +        return Failure(message=f"{label}: {e}")
100
101      velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
102      merge_gap = idx_cfg.dead_time_merge_gap
103      @@ -267,7 +283,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
104      ok, msg = runner.healthcheck()
105      if not ok:
106          logger.error("%s detector environment not ready: %s", label, msg)
107      -        return []
108      +        # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
109      +        # failed/retryable instead of writing an empty-but-"successful" result.
110      +        return Failure(
111      +            message=f"{label} detector environment not ready: {msg}",
112      +            is_recoverable=True,
113      +        )
114      logger.info("%s detector env OK: %s", label, msg)
115
116      # --- Phase 2: trajectory -> candidate rally windows ---
117      @@ -278,7 +299,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
118      csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
119      if not csv_path:
120          logger.error("%s produced no trajectory; aborting.", label)
121      -        return []
122      +        # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
123      +        # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
124      +        # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
125      +        # rather than a silent 0-segment "success" (the wf294 incident).
126      +        return Failure(
127      +            message=(
128      +                f"{label} detector produced no trajectory (timed out or aborted) ?
"
129      +                "detector failure, not a 0-rally video."
130      +            ),
131      +            is_recoverable=True,

```

```

131     +         )
132
133         points = runner.parse_trajectory_csv(csv_path)
134         logger.info(
135     @@ -387,7 +418,11 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
136         len(handoff_indices),
137     )
138
139     -         def process_candidate_window(wi: int, window: Dict[str, Any]) ->
List[dict]:
140     +         def process_candidate_window(
141     +             wi: int, window: Dict[str, Any]
142     +         ) -> Tuple[List[dict], bool]:
143     +             """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
144     +             verdict (clip extraction failed, or the provider returned
metrics['error'])."""
145             w_start = max(0, window["start"] - handoff_padding)
146             w_end = min(video_duration, window["end"] + handoff_padding)
147             w_dur = w_end - w_start
148     @@ -395,7 +430,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
149             if not extract_video_chunk(
150                 video_path, w_start, w_dur, w_path, reencode=True
151             ):
152                 return []
153     +             return [], True # extraction failed ? no verdict for this window
154             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
155             segments, metrics = provider.analyze_video(
156                 w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
157     @@ -403,7 +438,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
158             # B2: a provider failure returns [] with metrics["error"] set ? that is
159             # "we don't know", NOT "no rallies in this window". Surface it loudly
so a
160             # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
161     -             if metrics.get("error"):
162     +             errored = bool(metrics.get("error"))
163     +             if errored:
164                 logger.warning(
165                     "[Handoff %d] provider error ? window %d skipped: %s",
166                     wi + 1,
167     @@ -427,7 +463,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
168                 os.remove(w_path)
169                 except OSError:
170                     pass
171     -             return valid
172     +             return valid, errored
173
174             with concurrent.futures.ThreadPoolExecutor(
175                 max_workers=ai_max_workers
176     @@ -438,8 +474,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
177             )
178                 for wi in handoff_indices
179             ]
180             for future in futures:
181                 ai_segments.extend(future.result())
182             window_results = [future.result() for future in futures]
183             for valid, _errored in window_results:
184                 ai_segments.extend(valid)
185             handoff_failed = sum(1 for _valid, errored in window_results if errored)
186             try:
187                 os.rmdir(handoff_dir)
188             except OSError:
189     @@ -448,6 +486,18 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
190                 "Phase 3 completed. AI confirmed %d semantic play segments.",

```

```

191         len(ai_segments),
192     )
193     + # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
194     + # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
195     + # not a 0-rally match. A single window that RAN and returned no rallies is
a legit empty
196     + # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
197     + if handoff_indices and handoff_failed == len(handoff_indices):
198     +     msg = (
199     +         f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
200     +         "window(s) errored (provider error / clip extraction failed) ?
incomplete "
201     +         "run, not a 0-rally match."
202     +     )
203     +     logger.error(msg)
204     +     return Failure(message=msg, is_recoverable=True)
205
206     # --- Phase 4: DB population + candidate linking ---
207     segments_data = []
208     diff --git a/backend/pipeline/segmenters/yolo_hybrid.py
b/backend/pipeline/segmenters/yolo_hybrid.py
209     index d9334f1..2e2e1fe 100644
210     --- a/backend/pipeline/segmenters/yolo_hybrid.py
211     +++ b/backend/pipeline/segmenters/yolo_hybrid.py
212     @@ -22,11 +22,14 @@ import concurrent.futures
213     import logging
214     import os
215     import time
216     -from typing import Any, Dict, List, Tuple
217     +from typing import Any, Dict, List, Tuple, Union
218
219     from backend.pipeline.detectors.person_detector import PersonDetector
220     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
221     from backend.providers import provider_registry
222     +
223     +# Standardized Success|Failure contract: an unrunnable config is NOT a 0-rally result.
224     +from backend.results import Failure
225     from backend.sports import sport_registry
226     from backend.utils.paths import output_base
227     from backend.utils.video import extract_video_chunk, get_video_duration
228     @@ -63,9 +66,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
229     def _resolve_provider(self, video_path: str):
230     """Phase 0: resolve sport profile + AI provider (with API key).
231
232     - Returns the configured provider instance, or ``None`` to signal that
233     - ``process_video`` should early-return ``[]`` (bad sport, missing API key,
234     - or unknown provider). Behaviour is identical to the inlined version.
235     + Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
236     + or unknown provider) that ``process_video`` propagates ? so an unrunnable
config is marked
237     + FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
238     + mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
239     """
240     # Get sport profile config
241     sport_name = self.config.sport
242     @@ -73,7 +77,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
243     sport_registry.get(sport_name)
244     except KeyError as e:

```

```

245         logger.error(str(e))
246     -         return None
247     +         return Failure(str(e))
248
249         # Get AI provider and model config
250         provider_name = self.config.ai_provider
251     @@ -85,12 +89,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
252         if not api_key:
253             from backend.pipeline.segmenters._keyhint import api_key_hint
254
255     -         logger.error(
256     +         msg = (
257             f"{api_key_env_var} environment variable is not set! "
258             f"To run the {provider_name} segmenter, set your API key. "
259             + api_key_hint(api_key_env_var)
260         )
261     -         return None
262     +         logger.error(msg)
263     +         return Failure(msg, is_recoverable=True)
264
265         try:
266             provider = provider_registry.get(
267     @@ -98,7 +103,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
268             )
269         except KeyError as e:
270             logger.error(str(e))
271     -         return None
272     +         return Failure(str(e))
273
274         logger.info(
275             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
276     @@ -320,9 +325,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
277             chunk_duration: float,
278             handoff_padding: float,
279             ai_max_workers: int,
280     -         ) -> List[dict]:
281     +         ) -> "Union[List[dict], Failure]":
282             """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and
ask
283             the provider for semantic boundaries, in parallel. Verbatim move."""
284     +         the provider for semantic boundaries, in parallel.
285     +
286     +         Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
failed to
287     +         yield a verdict (provider error / clip extraction failed) ? an incomplete run
has zero
288     +         information and must NOT be recorded as a 0-rally "success" (the wf294
class)."""
289             gemini_segments = []
290             gemini_dir = os.path.join(
291                 output_base(self.config), f"chunks_{provider_name}_handoff"
292     @@ -333,7 +342,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
293                 f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
294             )
295
296     -         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
297     +         def process_candidate_window(
298     +             wi: int, window: Dict[str, Any]
299     +         ) -> Tuple[List[dict], bool]:
300     +             """Returns (valid_segments, errored) ? errored=True when this window
yielded NO verdict
301     +             (clip extraction failed, or the provider returned metrics['error'])."""
302             w_start = max(0, window["start"] - handoff_padding)

```

```

303         w_end = min(video_duration, window["end"] + handoff_padding)
304         w_dur = w_end - w_start
305     @@ -344,7 +357,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
306         video_path, w_start, w_dur, w_path, reencode=True
307     )
308     if not success:
309         -         return []
310         +         return [], True # extraction failed ? no verdict for this window
311
312     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
313     segments, metrics = provider.analyze_video(
314     @@ -353,7 +366,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
315         # B2: a provider failure returns [] with metrics["error"] set ? that is
316         # "we don't know", NOT "no rallies in this window". Surface it loudly so a
317         # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
318         -         if metrics.get("error"):
319         +         errored = bool(metrics.get("error"))
320         +         if errored:
321             logger.warning(
322                 "[Handoff %d] provider error ? window %d skipped: %s",
323                 wi + 1,
324     @@ -380,7 +394,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
325         except OSError:
326             pass
327
328         -         return valid_segments
329         +         return valid_segments, errored
330
331     with concurrent.futures.ThreadPoolExecutor(
332         max_workers=ai_max_workers
333     @@ -390,8 +404,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
334         for wi, w in enumerate(all_candidate_windows)
335     ]
336     # Iterate in submission order to preserve temporal ordering
337     -     for future in futures:
338     -         gemini_segments.extend(future.result())
339     +     window_results = [future.result() for future in futures]
340     +     for valid, _errored in window_results:
341     +         gemini_segments.extend(valid)
342     +     handoff_failed = sum(1 for _valid, errored in window_results if errored)
343
344     try:
345         os.rmdir(gemini_dir)
346     @@ -401,6 +417,17 @@ class HybridYoloSegmenter(BasePlaySegmenter):
347         logger.info(
348             f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
349         )
350     +     # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
351     +     # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
352     +     # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
353     +     if all_candidate_windows and handoff_failed == len(all_candidate_windows):
354     +         msg = (
355     +             f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
356     +             "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
357     +             "not a 0-rally match."
358     +         )
359     +         logger.error(msg)
360     +         return Failure(message=msg, is_recoverable=True)

```

```

361         return gemini_segments
362
363     def _populate_db(
364         @@ -456,19 +483,23 @@ class HybridYoloSegmenter(BasePlaySegmenter):
365
366         return segments_data
367
368     - def process_video(
369     + def process_video( # type: ignore[override]
370         self,
371         video_id: str,
372         video_path: str,
373         player_pool: List[str] = None,
374         max_frames: int = None,
375     - ) -> List[Dict[str, Any]]:
376     + ) -> "Union[List[Dict[str, Any]], Failure]":
377     +     # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
378     +     # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
379     +     # (the wf294 class). A genuine empty match still returns a bare []. Every
caller branches
380     +     # on isinstance(result, Failure).
381     self.person.lazy_load_model()
382
383     -     # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
384     +     # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
385     provider = self._resolve_provider(video_path)
386     -     if provider is None:
387     -         return []
388     +     if isinstance(provider, Failure):
389     +         return provider
390
391     # Re-read the config values needed downstream (pure deterministic reads).
392     idx_cfg = self.config.indexing
393     @@ -479,7 +510,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
394     video_duration = get_video_duration(video_path)
395     if video_duration <= 0:
396         logger.error("Invalid video duration.")
397     -     return []
398     +     # An unreadable/broken video, not a 0-rally match.
399     +     return Failure(
400     +         message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
401     +     )
402
403     chunk_duration = idx_cfg.chunk_duration
404     chunk_overlap = idx_cfg.chunk_overlap
405     @@ -537,8 +571,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
406     except Exception:
407         pass
408
409     -     # Phase 3: AI Provider Handoff
410     -     gemini_segments = self._run_ai_handoff(
411     +     # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
wf294).
412     +     handoff = self._run_ai_handoff(
413         video_id,
414         video_path,
415         video_duration,
416     @@ -551,9 +585,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
417         handoff_padding,
418         ai_max_workers,

```

```

419         )
420     +     if isinstance(handoff, Failure):
421     +         return handoff
422
423     # Phase 4: DB Population
424     -     return self._populate_db(video_id, gemini_segments, model_name)
425     +     return self._populate_db(video_id, handoff, model_name)
426
427
428     # Register the segmenter
429     diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
430     index 461d7ca..88c50f6 100644
431     --- a/backend/worker/__main__.py
432     +++ b/backend/worker/__main__.py
433     @@ -114,6 +114,42 @@ def _write_report_json(
434         )
435
436
437     +def _serialize_and_push_outputs(
438     +    scratch: str,
439     +    out_uri: str,
440     +    video_id: str,
441     +    segments: List[dict],
442     +    status: str,
443     +    storage_cfg: dict,
444     +    video_uri: str = "",
445     +) -> List[str]:
446     +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
447     +    push both to
448     +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
449     +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
450     +    timeout/abort leaves a
451     +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
452     +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
453     +    non-``success``
454     +    report as "no usable result" ? retry, and the restart-recovery poll waits for
455     +    report.json to
456     +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
457     +    out_local = os.path.join(scratch, "outputs", video_id)
458     +    os.makedirs(out_local, exist_ok=True)
459     +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
460     +    _write_report_json(
461     +        video_id,
462     +        segments,
463     +        status,
464     +        os.path.join(out_local, "report.json"),
465     +        video_uri=video_uri,
466     +    )
467     +    out_prefix = out_uri.rstrip("/")
468     +    pushed: List[str] = []
469     +    for fn in sorted(os.listdir(out_local)):
470     +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
471     +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
472     +        pushed.append(uri)
473     +    return pushed
474
475
476     def _run_startup_checks(config: Any) -> bool:
477     +    """M5 per-profile startup checks for the worker (the compute process, so it probes
478     +    CUDA
479     +    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
480     +    aborts
481     +    @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
482     +    )

```

```

478
479     def _fail(rc: int) -> int:
480         -         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
481         -         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
482         -         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
483         +         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
484         +         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
485         +         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
486         +         # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
487         +         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
488         +         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
489         +         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
490         +         try:
491         +             _serialize_and_push_outputs(
492         +                 scratch,
493         +                 args.out_uri,
494         +                 video_id,
495         +                 [],
496         +                 "failed",
497         +                 storage_cfg,
498         +                 str(getattr(args, "video_uri", "") or ""),
499         +             )
500         +         except Exception as e: # never mask the real failure on an artifact-push
hiccup
501         +             logger.warning("[worker] could not push failure artifacts: %s", e)
502         +             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
503         +             if getattr(args, "done_uri", None):
504         +                 _write_done_marker(
505         +                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
506 @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
507         +             )
508
509         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
510         -         out_local = os.path.join(scratch, "outputs", video_id)
511         -         os.makedirs(out_local, exist_ok=True)
512         -         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
513         -         _write_report_json(
514         +         pushed = _serialize_and_push_outputs(
515         +             scratch,
516         +             args.out_uri,
517         +             video_id,
518         +             segments,
519         +             status,
520         -             os.path.join(out_local, "report.json"),
521         -             video_uri=str(getattr(args, "video_uri", "") or ""),
522         +             storage_cfg,
523         +             str(getattr(args, "video_uri", "") or ""),
524         -         )
525
526         -         out_prefix = args.out_uri.rstrip("/")
527         -         pushed = []
528         -         for fn in sorted(os.listdir(out_local)):
529         -             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
530         -             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
531         -             pushed.append(uri)

```

```

532 -
533     print(
534         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
535     )
536     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
537     index 64909e7..c157ed5 100644
538     --- a/tests/test_tracknet_hybrid.py
539     +++ b/tests/test_tracknet_hybrid.py
540     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
541     from unittest.mock import MagicMock, patch
542
543     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
544     +from backend.results import Failure
545
546
547     def _window(start=1.0, end=4.0):
548     @@ -91,7 +92,7 @@ def test_pipeline_happy_path(
549
550
551     @_patched
552     -def test_pipeline_no_api_key_returns_empty(
553     +def test_pipeline_no_api_key_is_failure(
554         mock_runner_cls,
555         mock_probe,
556         mock_dur,
557     @@ -99,12 +100,16 @@ def test_pipeline_no_api_key_returns_empty(
558         mock_provider_registry,
559         mock_env,
560     ):
561     +     """A missing API key (with skip_ai_handoff=False) is an unrunnable misconfig ? a
Failure, NOT a
562     +     0-rally []. This is a wf294-class silent-failure the worker must mark failed, not
"success"."""
563         mock_env.return_value = None
564         mock_dur.return_value = 30.0
565         mock_runner_cls.return_value = _make_runner()
566
567         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
568     -     assert seg.process_video("v1", "clip.mp4") == []
569     +     res = seg.process_video("v1", "clip.mp4")
570     +     assert isinstance(res, Failure) and res.is_recoverable
571     +     assert "API_KEY" in res.message
572
573
574     @_patched
575     @@ -124,7 +129,10 @@ def test_pipeline_healthcheck_failure_aborts(
576
577         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
578         res = seg.process_video("v1", "clip.mp4")
579     -     assert res == []
580     +     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
581     +     # instead of writing an empty "success".
582     +     assert isinstance(res, Failure)
583     +     assert "environment not ready" in res.message
584     +     # Should bail before ever running inference.
585         mock_runner_cls.return_value.run_predict.assert_not_called()
586
587     @@ -144,7 +152,36 @@ def test_pipeline_no_trajectory_aborts(
588         mock_provider_registry.get.return_value = MagicMock()
589
590         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
591     -     assert seg.process_video("v1", "clip.mp4") == []

```

```

592 +     res = seg.process_video("v1", "clip.mp4")
593 +     # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
594 +     # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
595 +     assert isinstance(res, Failure)
596 +     assert "no trajectory" in res.message and res.is_recoverable
597 +
598 +
599 +@_patched
600 +def test_pipeline_zero_windows_is_legit_empty_not_failure(
601 +     mock_runner_cls,
602 +     mock_probe,
603 +     mock_dur,
604 +     mock_extract,
605 +     mock_provider_registry,
606 +     mock_env,
607 + ):
608 +     """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
609 +     parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
610 +     (0-rally "success"), NOT a Failure."""
611 +     mock_env.return_value = "dummy_key"
612 +     mock_dur.return_value = 30.0
613 +     mock_extract.return_value = True
614 +     mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
615 +     mock_provider_registry.get.return_value = MagicMock()
616 +
617 +     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
618 +     res = seg.process_video("v1", "clip.mp4")
619 +     assert res == [] and not isinstance(res, Failure)
620 +     # The detector DID run (a genuine empty, not an abort).
621 +     mock_runner_cls.return_value.run_predict.assert_called_once()
622
623
624     @_patched
625     diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
626     index 0080f4d..0270e41 100644
627     --- a/tests/test_trajectory_hybrid_equivalence.py
628     +++ b/tests/test_trajectory_hybrid_equivalence.py
629     @@ -14,6 +14,7 @@ import pytest
630     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
631     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
632     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
633     +from backend.results import Failure
634
635     TWINS = [
636         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
637     @@ -111,8 +112,10 @@ def test_provider_offsets_applied_identically(cls, family,
detector):
638
639     @pytest.mark.parametrize("cls,family,detector", TWINS)
640     def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
641     -     """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
642     -     treated as 'no rallies in this window'. The run still returns [] (no analysis
done)."""
643     +     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
644     +     silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
645     +     zero verdict, so process_video now returns a Failure (was a bare [] that looked

```

```

like a 0-rally
646 + match) ? the worker marks it failed/retryable instead of a silent empty
'success'. ""
647 import logging
648
649 provider = MagicMock()
650 @@ -139,13 +142,45 @@ def test_provider_failure_is_surfaced_not_silent(cls, family,
detector, caplog):
651     logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
652 ):
653     res = seg.process_video("v1", "clip.mp4")
654 - assert res == [] # no analysis for the failed window
655 + assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored
-> no verdict
656     assert any(
657         "provider error" in r.message and "generate failed" in r.message
658         for r in caplog.records
659     ), [r.message for r in caplog.records]
660
661
662 +@pytest.mark.parametrize("cls,family,detector", TWINS)
663 +def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
664 +     """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
665 +     no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
666 +     handed off; the first returns a rally, the second errors -> a Success with the
first rally. """
667 +     provider = MagicMock()
668 +     provider.analyze_video.side_effect = [
669 +         ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict
670 +         ([], {"error": "generate failed after retries: 503"}), # window 1: errored
671 +     ]
672 +     db = MagicMock()
673 +     db.add_candidate_segment.side_effect = [55, 56]
674 +     db.add_play_segment.return_value = 200
675 +     two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
676 +     with patch.object(
677 +         cls, "_build_runner", return_value=_runner(two_windows), {"weights_path":
"w"}))
678 +     ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
679 +         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
680 +         return_value=30.0,
681 +     ), patch(
682 +         "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
683 +         return_value=True,
684 +     ), patch(
685 +         "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
686 +     ) as preg, patch("os.environ.get", return_value="dummy_key"):
687 +         preg.get.return_value = provider
688 +         seg = cls(db, {"indexing": {}})
689 +         res = seg.process_video("v1", "clip.mp4")
690 +         assert not isinstance(res, Failure)
691 +         assert len(res) == 1 and res[0]["start_time"] == 0.5
692 +
693 +
694     def test_base_is_not_registered():
695         from backend.pipeline.segmenters.base import segmenter_registry
696
697     diff --git a/tests/test_worker.py b/tests/test_worker.py
698     index 96edbee..8fb5e16 100644
699     --- a/tests/test_worker.py
700     +++ b/tests/test_worker.py
701     @@ -10,6 +10,7 @@ import json
702     import pytest

```

```

703
704     from backend.config import load_config
705 +from backend.results import Failure
706     from backend.worker import __main__ as wmain
707
708     # ----- pure helpers
709 @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
710
711
712     class _EmptySeg:
713 -     """A segmenter that finds nothing ? the cold-start failure mode."""
714 +     """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
715
716         def __init__(self, db, config):
717             pass
718 @@ -200,6 +201,20 @@ class _EmptySeg:
719             return []
720
721
722 +class _FailSeg:
723 +     """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
724 +     trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
725 +
726         def __init__(self, db, config):
727             pass
728 +
729         def process_video(self, video_id, path, max_frames=None):
730             return Failure(
731                 message="WASB detector produced no trajectory (timed out or aborted)",
732                 is_recoverable=True,
733             )
734 +
735 +
736     def test_setup_logging_levels():
737         import logging
738
739 @@ -246,12 +261,112 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
740         assert "0 segments" in msgs and "vidZ" in msgs
741         assert "detector_impl=native" in msgs # surfaces the resolved detector
742         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
743 -     assert (
744 -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
745 -             "segment_count"
746 +     # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
747 +     # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
748 +     report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
749 +     assert report["segment_count"] == 0 and report["status"] == "success"
750 +
751 +
752 +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
753 +    tmp_path, monkeypatch
754 +):
755 +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
756 +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
757 +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
758 +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
759 +     monkeypatch.setattr(
760 +         wmain.validator_registry,

```

```

761 +         "run_all_with_context",
762 +         lambda path, cfg: ([_OKResult()], {}),
763 +     )
764 +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
765 +     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
766 +     video = tmp_path / "in.mp4"
767 +     video.write_bytes(b"FAKE")
768 +     out = tmp_path / "bucket"
769 +     done = tmp_path / "bucket" / "_dispatch" / "tok.done"
770 +
771 +     rc = wmain.main(
772 +         [
773 +             "infer",
774 +             "--video-uri",
775 +             str(video),
776 +             "--out-uri",
777 +             str(out),
778 +             "--scratch",
779 +             str(tmp_path / "s"),
780 +             "--segmenter",
781 +             "wasb_hybrid",
782 +             "--done-uri",
783 +             str(done),
784 +         ]
785 +     )
786 +     # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
787 +     assert rc == 3
788 +     # report.json is written as an explicit FAILURE, not a false empty "success".
789 +     report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
790 +     assert report["status"] == "failed" and report["segment_count"] == 0
791 +     assert report["segments"] == []
792 +     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
793 +     verdict.
794 +     marker = json.loads(done.read_text())
795 +     assert marker["returncode"] == 3 and marker["status"] == "failed"
796 +     assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
797 +
798 +     def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
799 + monkeypatch):
800 +         """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
801 + flows
802 + through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
803 + the two
804 + halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
805 + _FailSeg stub."""
806 +         from unittest.mock import MagicMock
807 +
808 +         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
809 +
810 +         runner = MagicMock()
811 +         runner.healthcheck.return_value = (True, "env ok")
812 +         runner.run_predict.return_value = None # WASB timed out / aborted: no trajectory
813 +         produced
814 +
815 +         # Force the real base process_video down to its no-trajectory abort without a
816 +         GPU/video.
817 +         monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
818 + (runner, {}))
819 +         monkeypatch.setattr(
820 +             WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
821 +         )
822 +         monkeypatch.setattr(
823 +             "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
824 + 30.0

```

```

817 + )
818 + monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
819 + monkeypatch.setattr(
820 +     wmain.validator_registry, "run_all_with_context", lambda path, cfg:
821 +     ([_OKResult()], {}))
822 + monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
823 + monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
824 +
825 + video = tmp_path / "in.mp4"
826 + video.write_bytes(b"FAKE")
827 + out = tmp_path / "bucket"
828 + done = tmp_path / "bucket" / "_dispatch" / "tok.done"
829 +
830 + rc = wmain.main(
831 +     [
832 +         "infer",
833 +         "--video-uri",
834 +         str(video),
835 +         "--out-uri",
836 +         str(out),
837 +         "--scratch",
838 +         str(tmp_path / "s"),
839 +         "--segmenter",
840 +         "wasb_hybrid",
841 +         "--done-uri",
842 +         str(done),
843 +         "--set",
844 +         "indexing.skip_ai_handoff=true", # no API key needed; isolate the detector
abort
845 +     ]
846 -     == 0
847 + )
848 + assert rc == 3
849 + report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
850 + assert report["status"] == "failed" and report["segment_count"] == 0
851 + marker = json.loads(done.read_text())
852 + assert marker["returncode"] == 3 and marker["status"] == "failed"
853
854
855 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
856 @@ -266,20 +381,30 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path,
monkeypatch):
857     video.write_bytes(b"X")
858     cfg = tmp_path / "config.json"
859     cfg.write_text("{}")
860 + out = tmp_path / "o"
861 + done = tmp_path / "o" / "_dispatch" / "tok.done"
862     rc = wmain.main(
863         [
864             "infer",
865             "--video-uri",
866             str(video),
867             "--out-uri",
868 -             str(tmp_path / "o"),
869 +             str(out),
870             "--config",
871             str(cfg),
872             "--scratch",
873             str(tmp_path / "s"),
874 +             "--done-uri",
875 +             str(done),
876         ]
877     )

```

```

878         assert rc == 2
879     +     # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
880     +     # success): report.json status="failed" + the done-marker rc 2 / status="failed".
881     +     report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
882     +     assert report["status"] == "failed" and report["segment_count"] == 0
883     +     marker = json.loads(done.read_text())
884     +     assert marker["returncode"] == 2 and marker["status"] == "failed"
885
886
887     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push
bundle ---
888     diff --git a/tests/test_yolo_hybrid.py b/tests/test_yolo_hybrid.py
889     index 38e78b2..09da6c2 100644
890     --- a/tests/test_yolo_hybrid.py
891     +++ b/tests/test_yolo_hybrid.py
892     @@ -11,6 +11,7 @@ import os
893     from unittest.mock import MagicMock, patch
894
895     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
896     +from backend.results import Failure
897
898
899     def _make_cap_mock(num_frames=300):
900     @@ -110,8 +111,10 @@ def test_yolo_hybrid_segmenter(
901     def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
902     mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
903     ):
904     -     """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
905     -     treated as 'no rallies'. The run still returns [] (no analysis done)."""
906     +     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
907     +     silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
908     +     zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
909     +     match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
910     import logging
911
912     mock_env_get.return_value = "dummy_api_key"
913     @@ -147,13 +150,49 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
914     logging.WARNING, logger="backend.pipeline.segmenters.yolo_hybrid"
915     ):
916     res = segmenter.process_video("v1", "dummy.mp4")
917     -     assert res == [] # no analysis for the failed window
918     +     assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored
-> no verdict
919     assert any(
920     "provider error" in r.message and "generate failed" in r.message
921     for r in caplog.records
922     ), [r.message for r in caplog.records]
923
924
925     +@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
926     +@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
927     +@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
928     +@patch("os.environ.get")
929     +def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(
930     +     mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
931     +):
932     +     """The boundary of F3: SOME windows errored but at least one RAN, so the run is a
Success (the
933     +     first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows

```

```

directly."""
934 + mock_env_get.return_value = "dummy_api_key"
935 + mock_get_dur.return_value = 30.0
936 + mock_extract.return_value = True
937 +
938 + mock_provider = MagicMock()
939 + mock_provider.analyze_video.side_effect = [
940 +     ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict
941 +     ([], {"error": "generate failed after retries: 503"}), # window 1: errored
942 + ]
943 + mock_provider_registry.get.return_value = mock_provider
944 +
945 + db_mock = MagicMock()
946 + db_mock.add_play_segment.return_value = "yolo_segment_1"
947 + two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
948 +
949 + segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
950 + segmenter.person.model = MagicMock() # short-circuit lazy model load
951 + with patch.object(
952 +     HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
953 + ), patch.object(
954 +     HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
955 + ):
956 +     res = segmenter.process_video("v1", "dummy.mp4")
957 +     assert not isinstance(res, Failure)
958 +     assert len(res) == 1 and res[0]["start_time"] == 0.5
959 +
960 +
961 + @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
962 + @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
963 + @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
964 + @@ -162,7 +201,8 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
965 +     def test_yolo_hybrid_no_api_key(
966 +         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
967 +     ):
968 +         """Segmenter should return empty when GEMINI_API_KEY is missing."""
969 +         """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that
the worker
970 +         would write as a silent status="success" (the wf294 class; yolo_hybrid is the
default segmenter)."""
971 +         mock_env_get.return_value = None
972 +
973 +         db_mock = MagicMock()
974 +         @@ -172,7 +212,8 @@ def test_yolo_hybrid_no_api_key(
975 +             segmenter.person.model = MagicMock() # short-circuit lazy model load
976 +
977 +             res = segmenter.process_video("v1", "dummy.mp4")
978 +             - assert res == []
979 +             + assert isinstance(res, Failure) and res.is_recoverable
980 +             + assert "API_KEY" in res.message
981 +
982 +
983 +         @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
984 +

```

4. assistant (2026-07-06T18:00:54.703Z)

I'll examine the source files to verify the completeness of these changes. Let me read the two segmenter files in full, plus the caller contract and related files.

5. user (2026-07-06T18:00:55.710Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a

```

```

4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20
21    import concurrent.futures
22    import logging
23    import os
24    import time
25    from typing import Any, Dict, List, Optional, Tuple, Union
26
27    import cv2
28
29    from backend.config.models import IndexingConfig
30    from backend.pipeline.detectors.base import DetectorRunner
31    from backend.pipeline.segmenters.base import BasePlaySegmenter
32    from backend.providers import provider_registry
33
34    # Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
35    from backend.results import Failure
36    from backend.sports import sport_registry
37    from backend.utils.paths import output_base
38    from backend.utils.video import extract_video_chunk, get_video_duration
39
40    logger = logging.getLogger(__name__)
41
42
43    def _fmt_ts(seconds: float) -> str:
44        seconds = max(0, int(seconds))
45        h, rem = divmod(seconds, 3600)
46        m, s = divmod(rem, 60)
47        return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50    class TrajectoryHybridSegmenter(BasePlaySegmenter):
51        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
52        boundaries."""
53
54        #: config section under `indexing` (velocity_threshold lives there), the
55        #: output subdir for trajectories, and the metadata detector-tag prefix.
56        family: str = ""
57        #: human-readable label used in log lines.
58        display_label: str = ""
59
60        def _build_runner(
61            self, idx_cfg: IndexingConfig
62        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
63            """Return (runner, detector-specific detection_params extras) for the default
64            (WSL) path."""
65            raise NotImplementedError
66
67        def _build_native_runner(
68            self, idx_cfg: IndexingConfig

```

```

67         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
68             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
69             native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
70             raise NotImplementedError(
71                 f"detector_impl='native' is not supported for the "
72                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
73                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
74             )
75
76     def _resolve_runner(
77         self, idx_cfg: IndexingConfig
78     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
79         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
80
81         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
82         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
83         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
84         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
86         (the WSL runner) so the default production path is unchanged
87         (``docs/DECOUPLED_COMPUTE``).
88         """
89         # Safety net: accept raw dicts from legacy / test callers.
90         if isinstance(idx_cfg, dict):
91             idx_cfg = IndexingConfig(**idx_cfg)
92         impl = (
93             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
94         ).lower()
95         if impl == "stub":
96             from backend.pipeline.detectors.stub_runner import (
97                 StubConfig,
98                 StubDetectorRunner,
99             )
100
101             logger.info(
102                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103                 self.display_label or "trajectory",
104             )
105             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
106                 "detector_impl": "stub"
107             }
108         if impl in ("native", "native_wasb", "wasb_native"):
109             logger.info(
110                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
111                 self.display_label or "trajectory",
112             )
113             return self._build_native_runner(idx_cfg)
114         return self._build_runner(idx_cfg)
115
116     @staticmethod
117     def _probe_fps_width(video_path: str) -> tuple:
118         """Return (fps, frame_width) for a video, with sensible fallbacks."""
119         cap = cv2.VideoCapture(video_path)
120         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122         cap.release()
123         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

```

```

124
125 @staticmethod
126 def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127     """Provider-reported exchange count, else a duration-based estimate.
128
129     One canonical implementation ? the twins had drifted (wasb relied on
130     ``int(None)`` raising into the fallback; same result, by accident).
131     """
132     shot_count = segment.get("shuttle_exchange_count")
133     if shot_count is None:
134         return max(3, int(duration / 0.8))
135     try:
136         return int(shot_count)
137     except (ValueError, TypeError):
138         return max(3, int(duration / 0.8))
139
140 # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
141 def _enrich_candidate_stats(
142     self,
143     cw: Dict[str, Any],
144     points: List[Any],
145     fps: float,
146     frame_width: float,
147     video_path: str,
148     idx_cfg: "IndexingConfig",
149 ) -> Dict[str, Any]:
150     """Stats dict persisted into ``candidate_segments.stats`` for one window. The
151     BASE
152     returns exactly the 4 motion keys, so the trajectory twins persist byte-
153     identical
154     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
155     features). ``points`` are the whole-video trajectory points
156     (TrajectoryPoint)."""
157     return {
158         "peak_velocity": cw["peak_velocity"],
159         "mean_velocity": cw["mean_velocity"],
160         "active_frame_count": cw["active_frame_count"],
161         "track_id_count": cw["track_id_count"],
162     }
163
164 def _select_for_handoff(
165     self,
166     windows: List[Dict[str, Any]],
167     window_stats: List[Dict[str, Any]],
168     idx_cfg: "IndexingConfig",
169 ) -> List[int]:
170     """Indices of the candidate windows that proceed to the AI handoff (and thus may
171     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
172     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
173     Persisted candidates are NOT affected ? they remain the full superset for
174     offline
175     re-scoring."""
176     return list(range(len(windows)))
177
178 def process_video( # type: ignore[override]
179     self,
180     video_id: str,
181     video_path: str,
182     player_pool: Optional[List[str]] = None,
183     max_frames: Optional[int] = None,
184 ) -> "Union[List[Dict[str, Any]], Failure]":
185     # Result contract (the ABC declares Success|Failure). Anything that means the
186     run could
187     # NOT actually complete ? an unrunnable config (unknown sport/provider, missing

```

```

API key),
183         # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
184         # not ready, or run_predict timing out / aborting with no trajectory ? returns a
``Failure``
185         # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
186         # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
187         # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
188         label = self.display_label
189         # --- Sport profile + AI provider wiring (identical across segmenters) ---
190         sport_name = self.config.sport
191         try:
192             sport_profile = sport_registry.get(sport_name)
193         except KeyError as e:
194             logger.error(str(e))
195             # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
196             return Failure(str(e))
197
198         idx_cfg = self.config.indexing
199         skip_ai = bool(idx_cfg.skip_ai_handoff)
200         provider_name = self.config.ai_provider
201         if skip_ai:
202             model_name = "local-noai"
203             logger.info(
204                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
205                 "be emitted as rallies; no %s handoff, no API key needed.",
206                 label,
207                 provider_name,
208             )
209         else:
210             model_name = self.config.ai_model
211             api_key_env_var = f"{provider_name.upper()}_API_KEY"
212             api_key = os.environ.get(api_key_env_var)
213             if not api_key:
214                 from backend.pipeline.segmenters._keyhint import api_key_hint
215
216                 msg = (
217                     f"{api_key_env_var} environment variable is not set! "
218                     f"To run the {provider_name} handoff, set your API key. "
219                     + api_key_hint(api_key_env_var)
220                 )
221                 logger.error(msg)
222                 # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
223                 # marked failed instead of a silent empty "success" (mirrors gemini; the
worker's
224                 # own 0-segment diagnostic names this missing-key case as a top cause).
225                 return Failure(msg, is_recoverable=True)
226             try:
227                 provider = provider_registry.get(
228                     provider_name, api_key=api_key, model_name=model_name
229                 )
230             except KeyError as e:
231                 logger.error(str(e))
232                 return Failure(str(e))
233
234         video_duration = get_video_duration(video_path)
235         if video_duration <= 0:
236             logger.error("Invalid video duration.")
237             # An unreadable/broken video, not a 0-rally match.
238             return Failure(
239                 message=f"{label}: invalid/unreadable video duration

```

```

({video_duration}).",
240     )
241     fps, frame_width = self._probe_fps_width(video_path)
242
243     # --- Runner config + windowing thresholds ---
244     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
245     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
246     # it delegates to the subclass _build_runner (the real WSL/native runner).
247     try:
248         runner, runner_params = self._resolve_runner(idx_cfg)
249     except NotImplementedError as e:
250         # e.g. detector_impl="native" for a family without a native runner
251         (tracknet, or
252         # fusion with a tracknet substrate). An unavailable detector is a failure,
253         not a
254         # 0-rally match ? refuse cleanly as a Failure instead of crashing or a false
255         success.
256         logger.error("%s: %s", label, e)
257         return Failure(message=f"{label}: {e}")
258
259     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
260     merge_gap = idx_cfg.dead_time_merge_gap
261     min_window_duration = idx_cfg.min_action_window_duration
262     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
263     IndexingConfig.
264     max_window_duration = idx_cfg.max_window_duration
265     window_min_split_gap = idx_cfg.window_min_split_gap
266     window_hard_cap = idx_cfg.window_hard_cap
267     window_inactivity_end = idx_cfg.window_inactivity_end
268     inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
269     inactivity_min_density = idx_cfg.inactivity_min_density
270     inactivity_temporal_density = idx_cfg.inactivity_temporal_density
271     window_blob_fallback = idx_cfg.window_blob_fallback
272     window_blob_factor = idx_cfg.window_blob_factor
273     handoff_padding = idx_cfg.ai_handoff_padding
274     ai_max_workers = idx_cfg.ai_max_workers
275     log_candidates = idx_cfg.log_yolo_candidates
276     store_candidates = idx_cfg.store_yolo_candidates
277
278     detection_params = {
279         "detector": self.name,
280         "velocity_thresh": velocity_thresh,
281         "merge_gap": merge_gap,
282         "min_action_window_duration": min_window_duration,
283         **runner_params,
284     }
285
286     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
287     ok, msg = runner.healthcheck()
288     if not ok:
289         logger.error("%s detector environment not ready: %s", label, msg)
290         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
291         the run
292         # failed/retryable instead of writing an empty-but-"successful" result.
293         return Failure(
294             message=f"{label} detector environment not ready: {msg}",
295             is_recoverable=True,
296         )
297     logger.info("%s detector env OK: %s", label, msg)
298
299     # --- Phase 2: trajectory -> candidate rally windows ---
300     # Trajectory detectors process the whole video in one pass (they carry
301     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
302     t_start = time.time()
303     out_dir = os.path.join(output_base(self.config), self.family)

```

```

299         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300     if not csv_path:
301         logger.error("%s produced no trajectory; aborting.", label)
302         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
303         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
304         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305         # rather than a silent 0-segment "success" (the wf294 incident).
306         return Failure(
307             message=(
308                 f"{label} detector produced no trajectory (timed out or aborted) ? "
309                 "detector failure, not a 0-rally video."
310             ),
311             is_recoverable=True,
312         )
313
314     points = runner.parse_trajectory_csv(csv_path)
315     logger.info(
316         "Parsed %d trajectory points (%d visible).",
317         len(points),
318         sum(1 for p in points if p.visible),
319     )
320
321     all_candidate_windows = runner.trajectory_to_action_windows(
322         points,
323         fps=fps,
324         frame_width=frame_width,
325         chunk_start=0.0,
326         velocity_thresh=velocity_thresh,
327         merge_gap=merge_gap,
328         min_window_duration=min_window_duration,
329         chunk_index=0,
330         max_window_duration=max_window_duration,
331         min_split_gap=window_min_split_gap,
332         window_hard_cap=window_hard_cap,
333         window_inactivity_end=window_inactivity_end,
334         inactivity_rally_thresh=inactivity_rally_thresh,
335         inactivity_min_density=inactivity_min_density,
336         inactivity_temporal_density=inactivity_temporal_density,
337         window_blob_fallback=window_blob_fallback,
338         window_blob_factor=window_blob_factor,
339     )
340     logger.info(
341         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
342         label,
343         time.time() - t_start,
344         len(all_candidate_windows),
345     )
346
347     if log_candidates:
348         for cw in all_candidate_windows:
349             logger.info(
350                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
351                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
352                 f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
353             )
354
355     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
356     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
357     # persistence and the handoff gate below.

```

```

358         window_stats = [
359             self._enrich_candidate_stats(
360                 cw, points, fps, frame_width, video_path, idx_cfg
361             )
362             for cw in all_candidate_windows
363         ]
364
365     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
366     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
367     if store_candidates:
368         for i, cw in enumerate(all_candidate_windows):
369             candidate_ids[i] = self.db.add_candidate_segment(
370                 video_id,
371                 detector=self.name,
372                 start_time=cw["start"],
373                 end_time=cw["end"],
374                 chunk_index=cw["chunk_index"],
375                 confidence=min(1.0, cw["peak_velocity"]),
376                 stats=window_stats[i],
377                 detection_params=detection_params,
378             )
379
380     if not all_candidate_windows:
381         return []
382
383     # --- Phase 3: AI provider handoff over padded candidate clips ---
384     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
385
386     # candidates above stay the full superset ? only the served play_segments are
387     # gated.
388     handoff_indices = self._select_for_handoff(
389         all_candidate_windows, window_stats, idx_cfg
390     )
391
392     ai_segments: List[dict] = []
393     if skip_ai:
394         # Fully-local: the selected candidate windows ARE the rallies ? emit them
395         # (no clip extraction, no upload). Boundaries are the local detector's; shot
396         # falls back to the duration-based estimate (no AI exchange count).
397         ai_segments = [
398             {
399                 "start_time": all_candidate_windows[wi]["start"],
400                 "end_time": all_candidate_windows[wi]["end"],
401                 "shuttle_exchange_count": None,
402                 "shot_count_confidence": "LocalNoAI",
403                 "_candidate_id": candidate_ids[wi],
404             }
405             for wi in handoff_indices
406         ]
407     else:
408         handoff_dir = os.path.join(
409             output_base(self.config), f"chunks_{provider_name}_handoff"
410         )
411         os.makedirs(handoff_dir, exist_ok=True)
412         logger.info(
413             "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies"
414             "(no AI handoff).",
415             len(ai_segments),
416         )
417         provider_name,

```

```

418         len(handoff_indices),
419     )
420
421     def process_candidate_window(
422         wi: int, window: Dict[str, Any]
423     ) -> Tuple[List[dict], bool]:
424         """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
425         verdict (clip extraction failed, or the provider returned
metrics['error'])."""
426         w_start = max(0, window["start"] - handoff_padding)
427         w_end = min(video_duration, window["end"] + handoff_padding)
428         w_dur = w_end - w_start
429         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
430         if not extract_video_chunk(
431             video_path, w_start, w_dur, w_path, reencode=True
432         ):
433             return [], True # extraction failed ? no verdict for this window
434             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
435             segments, metrics = provider.analyze_video(
436                 w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
437             )
438             # B2: a provider failure returns [] with metrics["error"] set ? that is
439             # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
440             # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
441             errored = bool(metrics.get("error"))
442             if errored:
443                 logger.warning(
444                     "[Handoff %d] provider error ? window %d skipped: %s",
445                     wi + 1,
446                     wi,
447                     metrics["error"],
448                 )
449             valid = []
450             for seg in segments:
451                 try:
452                     seg["start_time"] = float(seg["start_time"]) + w_start
453                     seg["end_time"] = float(seg["end_time"]) + w_start
454                     seg["_candidate_id"] = candidate_ids[wi]
455                     valid.append(seg)
456                 except (ValueError, TypeError, KeyError) as e:
457                     logger.warning(
458                         "Dropping segment with unparseable timestamps: %s ? %s",
459                         seg,
460                         e,
461                     )
462             try:
463                 os.remove(w_path)
464             except OSError:
465                 pass
466             return valid, errored
467
468     with concurrent.futures.ThreadPoolExecutor(
469         max_workers=ai_max_workers
470     ) as executor:
471         futures = [
472             executor.submit(
473                 process_candidate_window, wi, all_candidate_windows[wi]
474             )
475             for wi in handoff_indices
476         ]
477         window_results = [future.result() for future in futures]
478         for valid, _errored in window_results:

```

```

479         ai_segments.extend(valid)
480     handoff_failed = sum(1 for _valid, errored in window_results if errored)
481     try:
482         os.rmdir(handoff_dir)
483     except OSError:
484         pass
485     logger.info(
486         "Phase 3 completed. AI confirmed %d semantic play segments.",
487         len(ai_segments),
488     )
489     # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
490     # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
491     # not a 0-rally match. A single window that RAN and returned no rallies is a
legit empty
492     # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
493     if handoff_indices and handoff_failed == len(handoff_indices):
494         msg = (
495             f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
496             "window(s) errored (provider error / clip extraction failed) ?
incomplete "
497             "run, not a 0-rally match."
498         )
499         logger.error(msg)
500         return Failure(message=msg, is_recoverable=True)
501
502     # --- Phase 4: DB population + candidate linking ---
503     segments_data = []
504     for segment in ai_segments:
505         try:
506             start = float(segment["start_time"])
507             end = float(segment["end_time"])
508             dur = end - start
509             metadata = {
510                 "shot_count": self._shot_count_from(segment, dur),
511                 "shot_count_confidence": segment.get(
512                     "shot_count_confidence", "Unknown"
513                 ),
514                 "detector": f"{self.family}-hybrid-{model_name}",
515             }
516             segment_id = self.db.add_play_segment(
517                 video_id, start, end, None, None, metadata
518             )
519             if segment_id:
520                 candidate_id = segment.get("_candidate_id")
521                 if candidate_id is not None:
522                     self.db.link_candidate_to_segment(candidate_id, segment_id)
523                 segments_data.append(
524                     {
525                         "id": segment_id,
526                         "start_time": start,
527                         "end_time": end,
528                         "duration": dur,
529                         "metadata": metadata,
530                     }
531                 )
532             except Exception as e:
533                 logger.warning("Skipping invalid segment struct: %s", e)
534
535     return segments_data
536

```

6. user (2026-07-06T18:00:56.734Z)

```
1      """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2      (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4      Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5      (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6      can be consumed by the future fusion segmenter; this segmenter just orchestrates
7      chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9      Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11     detection uses torchvision's permissively-licensed (BSD) detectors and the
12     association/tracking that YOLO.track() used to bundle in is provided by ByteTrack
13     via the standalone Apache-2.0 ``trackers`` package (D13/P12 migration, #386 ?
14     previously ``supervision.ByteTrack``, removed upstream in supervision 0.30.0; see
15     ``person_detector.make_tracker`` for the behaviour-pinning details).
16
17     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
18     DB `detector` tags, even though no YOLO is involved anymore.
19     """
20
21     import concurrent.futures
22     import logging
23     import os
24     import time
25     from typing import Any, Dict, List, Tuple, Union
26
27     from backend.pipeline.detectors.person_detector import PersonDetector
28     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
29     from backend.providers import provider_registry
30
31     # Standardized Success|Failure contract: an unrunnable config is NOT a 0-rally result.
32     from backend.results import Failure
33     from backend.sports import sport_registry
34     from backend.utils.paths import output_base
35     from backend.utils.video import extract_video_chunk, get_video_duration
36
37     logger = logging.getLogger(__name__)
38
39
40     def _fmt_ts(seconds: float) -> str:
41         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
42         seconds = max(0, int(seconds))
43         h, rem = divmod(seconds, 3600)
44         m, s = divmod(rem, 60)
45         return f"{h:02d}:{m:02d}:{s:02d}"
46
47
48     class HybridYoloSegmenter(BasePlaySegmenter):
49         def __init__(self, db, config):
50             super().__init__(db, config)
51             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
52             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
53             self.person = PersonDetector(config)
54
55         @property
56         def name(self) -> str:
57             return "yolo_hybrid"
58
59         @property
60         def description(self) -> str:
61             return (
62                 "Two-stage pipeline: torchvision person detection + ByteTrack for fast "
```

```

63         "candidate filtering, then an AI provider for high-precision semantic
boundaries."
64     )
65
66     def _resolve_provider(self, video_path: str):
67         """Phase 0: resolve sport profile + AI provider (with API key).
68
69         Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
70         or unknown provider) that ``process_video`` propagates ? so an unrunnable config
is marked
71         FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
72         mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
73         """
74         # Get sport profile config
75         sport_name = self.config.sport
76         try:
77             sport_registry.get(sport_name)
78         except KeyError as e:
79             logger.error(str(e))
80             return Failure(str(e))
81
82         # Get AI provider and model config
83         provider_name = self.config.ai_provider
84         model_name = self.config.ai_model
85
86         # Get appropriate API key based on the provider
87         api_key_env_var = f"{provider_name.upper()}_API_KEY"
88         api_key = os.environ.get(api_key_env_var)
89         if not api_key:
90             from backend.pipeline.segmenters._keyhint import api_key_hint
91
92             msg = (
93                 f"{api_key_env_var} environment variable is not set! "
94                 f"To run the {provider_name} segmenter, set your API key. "
95                 + api_key_hint(api_key_env_var)
96             )
97             logger.error(msg)
98             return Failure(msg, is_recoverable=True)
99
100         try:
101             provider = provider_registry.get(
102                 provider_name, api_key=api_key, model_name=model_name
103             )
104         except KeyError as e:
105             logger.error(str(e))
106             return Failure(str(e))
107
108         logger.info(
109             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
110         )
111         return provider
112
113     def _collect_candidate_windows(
114         self,
115         video_id: str,
116         video_path: str,
117         video_duration: float,
118         idx_cfg: Dict[str, Any],
119         chunk_duration: float,
120         chunk_overlap: float,
121         velocity_thresh: float,

```

```

122         merge_gap: float,
123         frame_skip: int,
124         chunks_dir: str,
125     ) -> List[Dict[str, Any]]:
126         """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
127         sequential), then dedup-merge overlapping windows across chunks. Verbatim
128         move."""
129         step = chunk_duration - chunk_overlap
130         chunk_starts = []
131         t = 0.0
132         while t < video_duration:
133             chunk_starts.append(t)
134             t += step
135
136         num_chunks = len(chunk_starts)
137         logger.info(
138             f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
139             chunks."
140         )
141
142         all_candidate_windows = []
143
144         t_start = time.time()
145         prefetch_workers = idx_cfg.yolo_prefetch_workers
146
147         if num_chunks > 1 and prefetch_workers > 0:
148             # --- PIPELINED PROCESSING ---
149             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
150             # extraction is pure I/O. We pre-extract upcoming chunks in background
151             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
152             logger.info(
153                 f"Running detection Phase 2 with pipelined extraction
154                 ({prefetch_workers} prefetch workers)..."
155             )
156
157             extract_executor = concurrent.futures.ThreadPoolExecutor(
158                 max_workers=prefetch_workers
159             )
160
161             def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
162                 chunk_end = min(cs + chunk_duration, video_duration)
163                 actual_dur = chunk_end - cs
164                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
165                 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
166                 return (ci, cs, chunk_path, success)
167
168             # Submit all extractions upfront ? they'll queue and run as workers free up
169             extract_futures = [
170                 extract_executor.submit(_extract_chunk, ci, cs)
171                 for ci, cs in enumerate(chunk_starts)
172             ]
173
174             # Process results in order: wait for extraction, then run inference on GPU
175             for future in extract_futures:
176                 ci, chunk_start, chunk_path, success = future.result()
177                 chunk_end = min(chunk_start + chunk_duration, video_duration)
178                 logger.info(
179                     f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
180                     {chunk_end:.1f}s)..."
181                 )
182
183                 if not success:
184                     logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
185                     continue

```

```

183         windows = self.person.extract_action_windows_from_chunk(
184             chunk_path,
185             chunk_start,
186             velocity_thresh,
187             merge_gap,
188             frame_skip=frame_skip,
189             chunk_index=ci,
190         )
191         logger.info(
192             f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
193         )
194         all_candidate_windows.extend(windows)
195
196         try:
197             os.remove(chunk_path)
198         except Exception:
199             pass
200
201         extract_executor.shutdown(wait=False)
202     else:
203         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
204         for ci, chunk_start in enumerate(chunk_starts):
205             chunk_end = min(chunk_start + chunk_duration, video_duration)
206             actual_dur = chunk_end - chunk_start
207             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
208
209             logger.info(
210                 f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
211 {chunk_end:.1f}s)..."
212             )
213             success = extract_video_chunk(
214                 video_path, chunk_start, actual_dur, chunk_path
215             )
216             if not success:
217                 continue
218
219             windows = self.person.extract_action_windows_from_chunk(
220                 chunk_path,
221                 chunk_start,
222                 velocity_thresh,
223                 merge_gap,
224                 frame_skip=frame_skip,
225                 chunk_index=ci,
226             )
227             logger.info(
228                 f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
229             )
230             all_candidate_windows.extend(windows)
231
232             try:
233                 os.remove(chunk_path)
234             except Exception:
235                 pass
236
237     # Deduplicate overlapping candidate windows across chunks (chunks overlap by
238     # chunk_overlap, so the same rally can surface in two adjacent chunks).
239     def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
240         fa, fb = a["active_frame_count"], b["active_frame_count"]
241         total = fa + fb or 1
242         return {
243             "start": a["start"],
244             "end": max(a["end"], b["end"]),
245             "active_frame_count": fa + fb,
246             "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
247             # Frame-weighted mean so longer sub-windows dominate proportionally.

```

```

247         "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb)
248         / total,
249         "track_id_count": max(a["track_id_count"], b["track_id_count"]),
250         "chunk_index": a["chunk_index"],
251     }
252
253     if all_candidate_windows:
254         all_candidate_windows.sort(key=lambda w: w["start"])
255         merged_windows = [all_candidate_windows[0]]
256         for current in all_candidate_windows[1:]:
257             last = merged_windows[-1]
258             if current["start"] <= last["end"]: # Overlap
259                 merged_windows[-1] = _merge_stats(last, current)
260             else:
261                 merged_windows.append(current)
262         all_candidate_windows = merged_windows
263
264     logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
265     logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
266
267     return all_candidate_windows
268
269     def _persist_candidates(
270         self,
271         video_id: str,
272         all_candidate_windows: List[Dict[str, Any]],
273         detection_params: Dict[str, Any],
274         log_candidates: bool,
275         store_candidates: bool,
276     ) -> List[Any]:
277         """Phase 2b: per-rally console log + persist raw candidates for the fine-tuning
278         dataset. Returns window-index -> candidate_id (None where not stored). Verbatim
279         move."""
280         # Per-rally console log (final, full-video timestamps) for video cross-
281         verification.
282         if log_candidates:
283             for w in all_candidate_windows:
284                 logger.info(
285                     f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
286                     f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']} "
287                     f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
288                     f"tracks={w['track_id_count']}"
289                 )
290             # Persist raw candidates for the fine-tuning dataset. Map window index ->
291             candidate_id
292             # so confirmed AI segments can be linked back in Phase 4.
293             candidate_ids = [None] * len(all_candidate_windows)
294             if store_candidates:
295                 for i, w in enumerate(all_candidate_windows):
296                     stats = {
297                         "peak_velocity": w["peak_velocity"],
298                         "mean_velocity": w["mean_velocity"],
299                         "active_frame_count": w["active_frame_count"],
300                         "track_id_count": w["track_id_count"],
301                     }
302                     # Use peak velocity (capped at 1.0) as a coarse generic confidence
303                     score.
304                     confidence = min(1.0, w["peak_velocity"])
305                     candidate_ids[i] = self.db.add_candidate_segment(
306                         video_id,
307                         detector="yolo_hybrid",
308                         start_time=w["start"],
309                         end_time=w["end"],

```

```

307         chunk_index=w["chunk_index"],
308         confidence=confidence,
309         stats=stats,
310         detection_params=detection_params,
311     )
312
313     return candidate_ids
314
315     def _run_ai_handoff(
316         self,
317         video_id: str,
318         video_path: str,
319         video_duration: float,
320         all_candidate_windows: List[Dict[str, Any]],
321         candidate_ids: List[Any],
322         provider,
323         sport_profile,
324         provider_name: str,
325         chunk_duration: float,
326         handoff_padding: float,
327         ai_max_workers: int,
328     ) -> "Union[List[dict], Failure]":
329         """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
330         the provider for semantic boundaries, in parallel.
331
332         Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
333         failed to
334         yield a verdict (provider error / clip extraction failed) ? an incomplete run
335         has zero
336         information and must NOT be recorded as a 0-rally "success" (the wf294
337         class)."""
338         gemini_segments = []
339         gemini_dir = os.path.join(
340             output_base(self.config), f"chunks_{provider_name}_handoff"
341         )
342         os.makedirs(gemini_dir, exist_ok=True)
343
344         logger.info(
345             f"Starting Phase 3: AI Provider ({provider_name}) Validation on
346             {len(all_candidate_windows)} candidate windows..."
347         )
348
349         def process_candidate_window(
350             wi: int, window: Dict[str, Any]
351         ) -> Tuple[List[dict], bool]:
352             """Returns (valid_segments, errored) ? errored=True when this window yielded
353             NO verdict
354
355             (clip extraction failed, or the provider returned metrics['error'])."""
356             w_start = max(0, window["start"] - handoff_padding)
357             w_end = min(video_duration, window["end"] + handoff_padding)
358             w_dur = w_end - w_start
359
360             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
361             # Re-encode short handoff clips to avoid keyframe-boundary drift
362             success = extract_video_chunk(
363                 video_path, w_start, w_dur, w_path, reencode=True
364             )
365             if not success:
366                 return [], True # extraction failed ? no verdict for this window
367
368             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
369             segments, metrics = provider.analyze_video(
370                 w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
371             )
372             # B2: a provider failure returns [] with metrics["error"] set ? that is

```

```

367         # "we don't know", NOT "no rallies in this window". Surface it loudly so a
368         # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
369         errored = bool(metrics.get("error"))
370         if errored:
371             logger.warning(
372                 "[Handoff %d] provider error ? window %d skipped: %s",
373                 wi + 1,
374                 wi,
375                 metrics["error"],
376             )
377
378         # Coordinate mapping ? shift local timestamps back to full-video time
379         valid_segments = []
380         for segment in segments:
381             try:
382                 segment["start_time"] = float(segment["start_time"]) + w_start
383                 segment["end_time"] = float(segment["end_time"]) + w_start
384                 # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
385                 segment["_candidate_id"] = candidate_ids[wi]
386                 valid_segments.append(segment)
387             except (ValueError, TypeError, KeyError) as e:
388                 logger.warning(
389                     f"Dropping segment with unparseable timestamps: {segment} ? {e}"
390                 )
391
392             try:
393                 os.remove(w_path)
394             except OSError:
395                 pass
396
397         return valid_segments, errored
398
399     with concurrent.futures.ThreadPoolExecutor(
400         max_workers=ai_max_workers
401     ) as executor:
402         futures = [
403             executor.submit(process_candidate_window, wi, w)
404             for wi, w in enumerate(all_candidate_windows)
405         ]
406         # Iterate in submission order to preserve temporal ordering
407         window_results = [future.result() for future in futures]
408     for valid, _errored in window_results:
409         gemini_segments.extend(valid)
410     handoff_failed = sum(1 for _valid, errored in window_results if errored)
411
412     try:
413         os.rmdir(gemini_dir)
414     except OSError:
415         pass
416
417     logger.info(
418         f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
419     )
420     # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
421     # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
422     # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
423     if all_candidate_windows and handoff_failed == len(all_candidate_windows):
424         msg = (
425             f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}

```

```

candidate "
426         "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
427         "not a 0-rally match."
428     )
429     logger.error(msg)
430     return Failure(message=msg, is_recoverable=True)
431     return gemini_segments
432
433 def _populate_db(
434     self, video_id: str, gemini_segments: List[dict], model_name: str
435 ) -> List[Dict[str, Any]]:
436     """Phase 4: DB Population ? write confirmed AI segments, link them back to their
437     source candidate, and build the returned segments_data. Verbatim move."""
438     segments_data = []
439     for segment in gemini_segments:
440         try:
441             start = float(segment["start_time"])
442             end = float(segment["end_time"])
443             dur = end - start
444
445             shot_count = segment.get("shuttle_exchange_count")
446             if shot_count is None:
447                 shot_count = max(3, int(dur / 0.8))
448             else:
449                 try:
450                     shot_count = int(shot_count)
451                 except (ValueError, TypeError):
452                     shot_count = max(3, int(dur / 0.8))
453
454             metadata = {
455                 "shot_count": shot_count,
456                 "shot_count_confidence": segment.get(
457                     "shot_count_confidence", "Unknown"
458                 ),
459                 "detector": f"yolo-hybrid-{model_name}",
460             }
461
462             segment_id = self.db.add_play_segment(
463                 video_id, start, end, None, None, metadata
464             )
465             if segment_id:
466                 # Link back to the candidate that produced this segment. A candidate
467                 # may yield multiple AI segments (one-to-many); confirmed_segment_id
468                 # the last one ? the link table records "this candidate was
469                 confirmed".
470                 candidate_id = segment.get("_candidate_id")
471                 if candidate_id is not None:
472                     self.db.link_candidate_to_segment(candidate_id, segment_id)
473                 segments_data.append(
474                     {
475                         "id": segment_id,
476                         "start_time": start,
477                         "end_time": end,
478                         "duration": dur,
479                         "metadata": metadata,
480                     }
481                 )
482             except Exception as e:
483                 logger.warning(f"Skipping invalid segment struct: {e}")
484
485     return segments_data
486
487 def process_video( # type: ignore[override]

```

```

487         self,
488         video_id: str,
489         video_path: str,
490         player_pool: List[str] = None,
491         max_frames: int = None,
492     ) -> "Union[List[Dict[str, Any]], Failure]":
493         # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
494         # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
495         # (the wf294 class). A genuine empty match still returns a bare []. Every caller
branches
496         # on isinstance(result, Failure).
497         self.person.lazy_load_model()
498
499         # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
500         provider = self._resolve_provider(video_path)
501         if isinstance(provider, Failure):
502             return provider
503
504         # Re-read the config values needed downstream (pure deterministic reads).
505         idx_cfg = self.config.indexing
506         sport_profile = sport_registry.get(self.config.sport)
507         provider_name = self.config.ai_provider
508         model_name = self.config.ai_model
509
510         video_duration = get_video_duration(video_path)
511         if video_duration <= 0:
512             logger.error("Invalid video duration.")
513             # An unreadable/broken video, not a 0-rally match.
514             return Failure(
515                 message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
516             )
517
518         chunk_duration = idx_cfg.chunk_duration
519         chunk_overlap = idx_cfg.chunk_overlap
520         velocity_thresh = (
521             idx_cfg.yolo_velocity_threshold
522         ) # Fraction of frame width per second
523         merge_gap = idx_cfg.dead_time_merge_gap
524         frame_skip = idx_cfg.yolo_frame_skip
525         tracker_config = idx_cfg.yolo_tracker
526         handoff_padding = idx_cfg.ai_handoff_padding
527         ai_max_workers = idx_cfg.ai_max_workers
528         log_candidates = idx_cfg.log_yolo_candidates
529         store_candidates = idx_cfg.store_yolo_candidates
530
531         # Snapshot of the params that produced these detections ? stored alongside each
532         # candidate so a fine-tuning run can reproduce / correlate against them.
533         detection_params = {
534             "velocity_thresh": velocity_thresh,
535             "merge_gap": merge_gap,
536             "frame_skip": frame_skip,
537             "tracker": tracker_config,
538             "detector_model": idx_cfg.detector_model,
539             "detector_score_threshold": idx_cfg.detector_score_threshold,
540             "min_action_window_duration": idx_cfg.min_action_window_duration,
541         }
542
543         chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
544         os.makedirs(chunks_dir, exist_ok=True)
545
546         # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.

```

```

547         all_candidate_windows = self._collect_candidate_windows(
548             video_id,
549             video_path,
550             video_duration,
551             idx_cfg,
552             chunk_duration,
553             chunk_overlap,
554             velocity_thresh,
555             merge_gap,
556             frame_skip,
557             chunks_dir,
558         )
559
560         # Phase 2b: log + persist raw candidates (window index -> candidate_id).
561         candidate_ids = self._persist_candidates(
562             video_id,
563             all_candidate_windows,
564             detection_params,
565             log_candidates,
566             store_candidates,
567         )
568
569         try:
570             os.rmdir(chunks_dir)
571         except Exception:
572             pass
573
574         # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
575         wf294).
576         handoff = self._run_ai_handoff(
577             video_id,
578             video_path,
579             video_duration,
580             all_candidate_windows,
581             candidate_ids,
582             provider,
583             sport_profile,
584             provider_name,
585             chunk_duration,
586             handoff_padding,
587             ai_max_workers,
588         )
589         if isinstance(handoff, Failure):
590             return handoff
591
592         # Phase 4: DB Population
593         return self._populate_db(video_id, handoff, model_name)
594
595     # Register the segmenter
596     segmenter_registry.register(HybridYoloSegmenter)
597

```

7. assistant (2026-07-06T18:01:01.879Z)

Now let me examine the callers and the Failure/results contract, plus the worker code around `cmd_infer` and the ``_run_ai_handoff`` empty-candidate-windows path.

8. user (2026-07-06T18:01:02.904Z)

```

1     """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3     This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5     The goal is to make it impossible to confuse "no data / no rallies" with "something

```

```

actually failed"
6      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8      We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9      This is intentionally small and incremental ? we start by applying it to the segmenter
10     and provider layers (the biggest sources of silent [] returns today).
11
12     The types are generic where it makes sense. Callers can still access .value on Success
13     or .message on Failure.
14
15     This module has no dependencies on the rest of the app.
16     """
17
18     from __future__ import annotations
19
20     from dataclasses import dataclass, field
21     from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
23     T = TypeVar("T")
24
25
26     @dataclass
27     class Success(Generic[T]):
28         """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30         value: T
31         warnings: List[str] = field(default_factory=list)
32
33         def is_success(self) -> bool:
34             return True
35
36         def unwrap(self) -> T:
37             """Return the value. Only safe to call if you already know it's a Success."""
38             return self.value
39
40
41     @dataclass
42     class Failure:
43         """Represents a failure with a human-readable message and optional details."""
44
45         message: str
46         details: Optional[dict[str, Any]] = None
47         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49         def is_success(self) -> bool:
50             return False
51
52         def unwrap(self) -> Any:
53             """Raise to make misuse obvious in code that assumes success."""
54             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
62
63     def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64         """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66         - ``Failure``          -> ``(None, failure)``
67         - ``Success(value=v)`` -> ``(v, None)``

```

```

68         - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70
71     This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72     that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73     on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74     (Success | Failure | legacy bare value)."""
75     if isinstance(result, Failure):
76         return None, result
77     value = result.value if hasattr(result, "value") else result
78     return value, None
79

```

9. user (2026-07-06T18:01:03.847Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16
17    from __future__ import annotations
18
19    import argparse
20    import csv
21    import json
22    import logging
23    import os
24    import re
25    import sys
26    import tempfile
27    from typing import Any, List
28
29    from backend import storage
30    from backend.compute import ComputeJobSpec, get_compute_backend
31    from backend.config import (
32        StartupCheckError,
33        enforce_startup_checks,
34        load_config,
35        probe_cuda_available,
36    )
37    from backend.infrastructure.database import Database
38    from backend.pipeline.segmenters.base import segmenter_registry
39    from backend.pipeline.validators.base import registry as validator_registry
40    from backend.reporting import emit_indexer_report
41    from backend.results import Failure
42    from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver

```

```

43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63         indexing.skip_ai_handoff=true)."""
64         for kv in sets or []:
65             key, sep, val = kv.partition("=")
66             if not sep:
67                 raise ValueError(f"--set expects k=v, got {kv!r}")
68             parts = key.split(".")
69             obj = config
70             for p in parts[:-1]:
71                 obj = getattr(obj, p)
72             setattr(obj, parts[-1], _coerce(val))
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76         friendly
77         artifact (same schema as the rally-annotator labels)."""
78         with open(path, "w", newline="") as f:
79             w = csv.writer(f)
80             w.writerow(["start", "end", "shot_count", "ending_reason"])
81             for s in segments:
82                 w.writerow(
83                     [
84                         f"{float(s.get('start_time', 0.0)):.3f}",
85                         f"{float(s.get('end_time', 0.0)):.3f}",
86                         s.get("shot_count", ""),
87                         s.get("ending_reason", s.get("shot_count_confidence", "")),
88                     ]
89                 )
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [

```

```

105         {
106             "start": float(s.get("start_time", 0.0)),
107             "end": float(s.get("end_time", 0.0)),
108         }
109         for s in segments
110     ],
111 },
112 f,
113     indent=2,
114 )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a

```

```

161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
180         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
181         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
182         from backend.infrastructure.execution_policy import PolicyTimeout
183
184         spec = ComputeJobSpec(
185             video_uri=args.video_uri,
186             out_uri=args.out_uri,
187             segmenter=args.segmenter,
188             config_overrides=tuple(args.set or ()),
189         )
190         try:
191             result = compute.run_infer(spec)
192         except PolicyTimeout as e:
193             logger.warning(
194                 "[worker] remote compute did not complete (no marker within budget): %s", e
195             )
196             return 4
197         logger.info(
198             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
199             result.video_id,
200             result.returncode,
201             result.outputs_uri,
202         )
203         return result.returncode
204
205
206     def _write_done_marker(
207         done_uri: str,
208         video_id: str,
209         returncode: int,
210         segment_count: int,
211         status: str,
212         storage_cfg: dict,
213         scratch: str,
214     ) -> None:
215         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
216         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""

```

```

217     try:
218         marker = {
219             "video_id": video_id,
220             "returncode": returncode,
221             "segment_count": segment_count,
222             "status": status,
223         }
224         local = os.path.join(scratch, "_done.json")
225         with open(local, "w", encoding="utf-8") as f:
226             json.dump(marker, f)
227         storage.put(local, done_uri, config=storage_cfg)
228         logger.info("[worker] wrote completion marker -> %s", done_uri)
229     except Exception as e: # never fail a good run because the marker push hiccuped
230         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
231
232
233     def cmd_infer(args: argparse.Namespace) -> int:
234         config = load_config(args.config) if args.config else load_config()
235         _apply_overrides(config, args.set)
236         if not _run_startup_checks(config):
237             return 2
238
239         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
240         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
241         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
242         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
243         compute = get_compute_backend(config)
244         if compute is not None:
245             return _dispatch_remote(compute, args)
246
247         storage_cfg = config.storage.model_dump()
248
249         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
250         os.makedirs(scratch, exist_ok=True)
251         config.output.output_dir = scratch
252
253         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
254         local_video = storage.fetch(
255             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
256         )
257         print(f"[worker] pulled {args.video_uri} -> {local_video}")
258
259         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
260         video_id = compute_video_id(local_video)
261
262         # 3. VALIDATE (same registry as the main CLI).
263         val_results, val_context = validator_registry.run_all_with_context(
264             local_video, config
265         )
266         failures = [r for r in val_results if not r.passed]
267         db = Database(os.path.join(scratch, config.output.db_name))
268         db.add_video(
269             video_id,
270             local_video,
271             "failed" if failures else "processed",
272             [r.model_dump() for r in val_results],
273         )
274
275     def _fail(rc: int) -> int:

```

```

276         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
277         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
278         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283         try:
284             _serialize_and_push_outputs(
285                 scratch,
286                 args.out_uri,
287                 video_id,
288                 [],
289                 "failed",
290                 storage_cfg,
291                 str(getattr(args, "video_uri", "") or ""),
292             )
293         except Exception as e: # never mask the real failure on an artifact-push hiccup
294             logger.warning("[worker] could not push failure artifacts: %s", e)
295         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
296         if getattr(args, "done_uri", None):
297             _write_done_marker(
298                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299             )
300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313         seg_name = args.segmenter or config.indexing.default_segmenter
314         segmenter_cls = segmenter_registry.get(seg_name)
315         if not segmenter_cls:
316             print(
317                 f"[worker] unknown segmenter: {seg_name!r} "
318                 f"(have {list(segmenter_registry.list_available())})"
319             )
320             return _fail(2)
321         segmenter_actual = seg_name
322         result = segmenter_cls(db, config).process_video(
323             video_id, local_video, max_frames=args.max_frames
324         )
325         segmentation_ran = True
326         if isinstance(result, Failure):
327             print(f"[worker] segmenter FAILED: {result.message}")
328             return _fail(3)
329         segments = result.value if hasattr(result, "value") else result
330         status = "success"
331         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332         # empty match ? the cold-start failure mode (the substrate detector yielded no

```

```

usable
333         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334         if not segments:
335             detector_impl = (
336                 os.environ.get("RALLY_DETECTOR_IMPL")
337                 or getattr(config.indexing, "detector_impl", None)
338                 or "auto"
339             )
340             logger.warning(
341                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347                 "detections, or the input resolution differs from the tuned proxy
resolution. "
348                 "Re-run with -v for the native command + frame counts.",
349                 video_id,
350                 segmenter_actual,
351                 detector_impl,
352             )
353         finally:
354             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355             emit_indexer_report(
356                 db=db,
357                 video_id=video_id,
358                 video_path=local_video,
359                 config=config,
360                 val_results=val_results,
361                 val_context=val_context,
362                 segmenter_requested=args.segmenter,
363                 segmenter_actual=segmenter_actual,
364                 was_reingest=False,
365                 segmentation_ran=segmentation_ran,
366             )
367
368         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369         pushed = _serialize_and_push_outputs(
370             scratch,
371             args.out_uri,
372             video_id,
373             segments,
374             status,
375             storage_cfg,
376             str(getattr(args, "video_uri", "") or ""),
377         )
378
379         print(
380             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
381         )
382         for u in pushed:
383             print("    ", u)
384
385         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
386         # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker

```

```

FAILURES (rc 2/3)
387         # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
388         # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
389         if getattr(args, "done_uri", None):
390             _write_done_marker(
391                 args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
392             )
393         return 0
394
395
396         #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
397         #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
398         _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
399
400         #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
401         #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
402         #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
403         _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
404
405
406         def _label_clip_name(fname: str) -> str:
407             """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
408
409             Strips both the `.rallies.csv` / `.csv` suffix and an optional leading ISO-date
prefix, so a
410             canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
411
412             * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-
stamped)
413             * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same
stem)
414             * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of
the name, kept)
415             * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in
<name> survive)
416
417             Without the date strip, `--trajectory-source detector` can't match a dated label
to its bare
418             video stem, so every clip is skipped and train aborts with `<2 videos`
(DATA_IN_GCS §7b #1).
419             """
420             name = fname.replace(".rallies.csv", "").replace(".csv", "")
421             return _LABEL_DATE_PREFIX.sub("", name)
422
423
424         def _probe_video_fps_width(path: str):
425             """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
426
427             The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
428             fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
429             cv2's `_probe_fps_width` silently defaults to (30, 1280) on a read miss ? fine for
the
430             inference report (it flags the fallback) but corrupting for REAL training data. So
here we
431             probe with ffprobe (same tool as `get_video_duration`) and return None so the
caller SKIPS

```

```

432         rather than guesses.
433         """
434         import json
435         import subprocess
436
437         try:
438             out = subprocess.check_output(
439                 [
440                     ffprobe_bin(),
441                     "-v",
442                     "error",
443                     "-select_streams",
444                     "v:0",
445                     "-show_entries",
446                     "stream=r_frame_rate,width",
447                     "-of",
448                     "json",
449                     path,
450                 ],
451                 stderr=subprocess.DEVNULL,
452             ).decode()
453             st = (json.loads(out).get("streams") or [{}])[0]
454             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
455             fps = float(num) / float(den or "1")
456             width = float(st.get("width") or 0)
457             if fps > 0 and width > 0:
458                 return fps, width
459         except (
460             subprocess.SubprocessError,
461             ValueError,
462             KeyError,
463             OSError,
464             json.JSONDecodeError,
465         ):
466             pass
467         return None
468
469
470 def _resolve_train_detector(args: argparse.Namespace, config: Any):
471     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
472     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
473     box,
474     stub=CPU mock). Returns the runner, or prints an error + returns None.
475
476     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
477     the
478     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
479     has
480     no shuttle detector and is rejected.
481     """
482     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
483
484     seg_cls = segmenter_registry.get(args.segmenter)
485     if not seg_cls:
486         print(
487             f"[worker] unknown segmenter: {args.segmenter!r} "
488             f"(have {list(segmenter_registry.list_available())})"
489         )
490         return None
491     seg = seg_cls(None, config)
492     if not isinstance(seg, TrajectoryHybridSegmenter):
493         print(
494             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
495             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
```

```

494         return None
495     try:
496         runner, _ = seg._resolve_runner(config.indexing)
497     except NotImplementedError as e:
498         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
499         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
500         traceback.
501         print(f"[worker] detector not available: {e}")
502         return None
503     ok, msg = runner.healthcheck()
504     if not ok:
505         print(f"[worker] detector environment not ready: {msg}")
506         return None
507     print(f"[worker] detector ready ({args.segmenter}): {msg}")
508     return runner
509
510 def cmd_train(args: argparse.Namespace) -> int:
511     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
512     shell out
513     to training.gen0.harness (backend must NOT import training) -> push the artifact
514     bundle.
515     Two trajectory sources (the train pipeline + harness are identical either way):
516     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
517     synthetic
518     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
519     CI.
520     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
521     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
522     This
523     is the M3.5 "first real training" path; only the trajectory source flips.
524     """
525     import subprocess
526
527     config = load_config(args.config) if args.config else load_config()
528     _apply_overrides(config, args.set)
529     if not _run_startup_checks(config):
530         return 2
531     storage_cfg = config.storage.model_dump()
532
533     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
534     labels_dir = os.path.join(scratch, "labels")
535     traj_dir = os.path.join(scratch, "trajectories")
536     videos_dir = os.path.join(scratch, "videos")
537     os.makedirs(labels_dir, exist_ok=True)
538     os.makedirs(traj_dir, exist_ok=True)
539
540     corpus = args.corpus_uri.rstrip("/")
541     label_uris = sorted(
542         u
543         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
544         if u.lower().endswith(".csv")
545     )
546     if len(label_uris) < 2:
547         print(
548             f"[worker] need >=2 labeled videos for leave-one-video-out; found
549             {len(label_uris)} "
550             f"under {corpus}/labels/"
551         )
552         return 2
553
554     # Real-detector source: resolve the runner once + index the bucket's videos/ by
555     stem.
556     runner = None

```

```

551     video_by_stem: dict = {}
552     if args.trajectory_source == "detector":
553         os.makedirs(videos_dir, exist_ok=True)
554         runner = _resolve_train_detector(args, config)
555         if runner is None:
556             return 2
557         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
558             vname = storage.parse_uri(vuri).name
559             if not vname.lower().endswith(_VIDEO_EXTS):
560                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
561             video_by_stem[os.path.splitext(vname)[0]] = vuri
562
563     print(f"[worker] trajectory source: {args.trajectory_source}")
564     specs: List[dict] = []
565     for uri in label_uris:
566         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
567         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
568         local_label = storage.fetch(
569             uri, os.path.join(labels_dir, fname), config=storage_cfg
570         )
571         if args.trajectory_source == "detector":
572             vuri = video_by_stem.get(name)
573             if not vuri:
574                 print(
575                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
576                 )
577                 continue
578             local_video = storage.fetch(
579                 vuri,
580                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
581                 config=storage_cfg,
582             )
583             video_id = compute_video_id(local_video)
584             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
585             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
586             meta = _probe_video_fps_width(local_video)
587             if meta is None:
588                 print(
589                     f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
590                 )
591                 continue
592             fps, frame_width = meta
593             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
594             if not traj:
595                 print(
596                     f"[worker] detector produced no trajectory for {name!r} ? skipping."
597                 )
598                 continue
599             specs.append(
600                 {
601                     "name": name,
602                     "trajectory": traj,
603                     "labels": local_label,
604                     "fps": fps,
605                     "frame_width": frame_width,
606                 }
607             )
608         else:
609             from backend.pipeline.detectors.stub_runner import (
610                 synth_trajectory_from_labels,
611             )

```

```

612
613     traj = os.path.join(traj_dir, f"{name}_traj.csv")
614     synth_trajectory_from_labels(
615         local_label, traj, fps=args.fps, frame_width=args.frame_width
616     )
617     specs.append(
618         {
619             "name": name,
620             "trajectory": traj,
621             "labels": local_label,
622             "fps": args.fps,
623             "frame_width": args.frame_width,
624         }
625     )
626
627     if len(specs) < 2:
628         print(
629             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
630         )
631         return 2
632     manifest_path = os.path.join(scratch, "manifest.json")
633     with open(manifest_path, "w", encoding="utf-8") as f:
634         json.dump(specs, f, indent=2)
635     src_label = (
636         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
637     )
638     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
639
640     out_dir = os.path.join(scratch, "artifacts")
641     cmd = [
642         sys.executable,
643         "-m",
644         "training.gen0.harness",
645         "--manifest",
646         manifest_path,
647         "--out",
648         out_dir,
649         "--version",
650         args.version,
651     ]
652     if args.floor:
653         floor_local = (
654             storage.fetch(
655                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
656             )
657             if "://" in args.floor
658             else args.floor
659         )
660         cmd += ["--floor", floor_local]
661     print("[worker] training:", " ".join(cmd))
662     rc = subprocess.run(cmd).returncode
663     if rc != 0:
664         print(f"[worker] gen0 harness failed (rc={rc})")
665         return rc
666
667     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
668     bundle = os.path.join(out_dir, args.version)
669     out_prefix = args.out_uri.rstrip("/")
670     pushed = []
671     for fn in sorted(os.listdir(bundle)):
672         uri = f"{out_prefix}/weights/{args.version}/{fn}"
673         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
674         pushed.append(uri)
675     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")

```

```

676         for u in pushed:
677             print(" ", u)
678         return 0
679
680
681 def build_parser() -> argparse.ArgumentParser:
682     p = argparse.ArgumentParser(
683         prog="python -m backend.worker",
684         description="Storage-aware worker: pull ? run pipeline ? push.",
685     )
686     p.add_argument(
687         "-v",
688         "--verbose",
689         action="store_true",
690         help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
691     )
692     sub = p.add_subparsers(dest="cmd", required=True)
693
694     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
695     pi.add_argument(
696         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
697     )
698     pi.add_argument(
699         "--out-uri",
700         required=True,
701         help="output prefix (outputs/<video_id>/? is appended)",
702     )
703     pi.add_argument(
704         "--segmenter", default=None, help="default: indexing.default_segmenter"
705     )
706     pi.add_argument(
707         "--config", default=None, help="config.json path (default: ./config.json)"
708     )
709     pi.add_argument(
710         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
711     )
712     pi.add_argument("--max-frames", type=int, default=None)
713     pi.add_argument(
714         "--done-uri",
715         default=None,
716         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
717         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
718         "Default-OFF: omit it for the normal local run.",
719     )
720     pi.add_argument(
721         "--set",
722         action="append",
723         default=[],
724         metavar="k=v",
725         help="config override, e.g. --set indexing.skip_ai_handoff=true",
726     )
727     pi.set_defaults(func=cmd_infer)
728
729     pt = sub.add_parser(
730         "train",
731         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
732     )
733     pt.add_argument(
734         "--corpus-uri",
735         required=True,
736         help="prefix containing labels/ (+ videos/ for "
737         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
738     )
739     pt.add_argument(

```

```

740         "--out-uri",
741         required=True,
742         help="output prefix (weights/<version>/ is appended)",
743     )
744     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
745     pt.add_argument(
746         "--trajectory-source",
747         choices=["synth", "detector"],
748         default="synth",
749         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
750         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
751         "(needs videos/ in the corpus; honours detector_impl).",
752     )
753     pt.add_argument(
754         "--segmenter",
755         default="wasb_hybrid",
756         help="trajectory hybrid whose detector is used for --trajectory-source",
757     )
758     pt.add_argument(
759         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
760     )
761     pt.add_argument("--config", default=None)
762     pt.add_argument("--scratch", default=None)
763     pt.add_argument(
764         "--fps",
765         type=float,
766         default=30.0,
767         help="synth trajectory fps (detector probes the video)",
768     )
769     pt.add_argument(
770         "--frame-width",
771         type=float,
772         default=1280.0,
773         help="synth trajectory frame width (detector probes the video)",
774     )
775     pt.add_argument("--set", action="append", default=[], metavar="k=v")
776     pt.set_defaults(func=cmd_train)
777     return p
778
779
780     def _setup_logging(verbose: bool) -> None:
781         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
782         diagnostics are
783         VISIBLE ? the worker previously ran with no logging config, so every
784         segmenter/detector log
785         (detector choice, 'Running native WASB inference', availability failures) was
786         silently dropped.
787         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
788         (the CLI
789         entry), not at import, so importing the module never reconfigures a caller's
790         logging."""
791         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
792         library
793         # configured logging at import), which would leave the level at WARNING and re-
794         suppress the
795         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
796         verbosity.
797         logging.basicConfig(
798             level=logging.DEBUG if verbose else logging.INFO,
799             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
800             datefmt="%H:%M:%S",
801             force=True,
802         )

```

```

796
797     def main(argv=None) -> int:
798         args = build_parser().parse_args(argv)
799         _setup_logging(getattr(args, "verbose", False))
800         return args.func(args)
801
802
803     if __name__ == "__main__":
804         sys.exit(main())
805

```

10. assistant (2026-07-06T18:01:10.397Z)

The core changes look coherent. Now let me examine the other callers (`main.py:1557`, `orchestration.py:806`) and the fusion segmenter, and check for any remaining `is None` checks or callers that assume `[]`.

11. user (2026-07-06T18:01:12.530Z)

```

backend\results.py:72:     that forget it (e.g. ``len(process_video(...))``) crash with
``TypeError: ... has no len()``
backend\main.py:905: def process_video(req: ProcessRequest, request: Request):
backend\main.py:1552:         result = segmenter.process_video(
backend\orchestration.py:795:         result = segmenter.process_video(
backend\worker\_main_.py:322:         result = segmenter_cls(db, config).process_video(
backend\api\jobs.py:7: per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
backend\api\jobs.py:32:     """ATOMIC idempotent-submit hook for backend.main.process_video (PR
#481 review): delegates to
backend\pipeline\segmenters\base.py:44:     def process_video(
backend\pipeline\segmenters\gemini.py:85:         """Resolve the sport profile, provider, and API
key (the process_video preamble).
backend\pipeline\segmenters\gemini.py:620:     def process_video(
backend\pipeline\segmenters\motion.py:23:     def process_video(
backend\pipeline\segmenters\motion.py:58:         before returning. This is the detection half of
``process_video`` ? it
backend\pipeline\segmenters\trajectory_hybrid.py:174:     def process_video( # type:
ignore[override]
backend\pipeline\segmenters\yolo_hybrid.py:70:         or unknown provider) that
``process_video`` propagates ? so an unrunnable config is marked
backend\pipeline\segmenters\yolo_hybrid.py:486:     def process_video( # type: ignore[override]
backend\eval\served_gate_a.py:13: verification reflects what ``process_video`` actually hands to
Gemini.
backend\eval\served_gate_a.py:38: # TrajectoryHybridSegmenter.process_video reads for the served
fusion run).
backend\eval\served_gate_a.py:84:     # call the pure decision seams ? no process_video, no DB,
no provider ? exactly as the
backend\eval\windowing.py:7: (``TrajectoryHybridSegmenter.process_video``) windows COARSELY
(``vt=0.02, merge_gap=2.0,
backend\eval\windowing.py:111:     ``TrajectoryHybridSegmenter.process_video`` reads exactly
these three:
backend\eval\windowing.py:164:     Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob
reads exactly, against an

```

12. user (2026-07-06T18:01:13.002Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,

```

```

9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10      is explicitly enabled (`indexing.fusion.cue_flags.person`), the person-cue features
11      (`fusion_features`), persisting them into `candidate_segments.stats` for the learned
12      fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13      torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14      model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15      via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - `_select_for_handoff` ? optional served Gate A/B: when
`indexing.fusion.min_crossings`
17      (and, with the person cue, `min_players`) are raised above 0, sub-threshold windows
are
18      dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19      **Persisted candidates remain the full superset** regardless, so the offline
experiment
20      harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as `fusion_hybrid`, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28
29      from typing import Any, Dict, List, Optional, Tuple
30
31      from backend.config.models import FusionConfig, IndexingConfig
32      from backend.pipeline.detectors import fusion_features as ff
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors.base import DetectorRunner
35      from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36      from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
37      from backend.pipeline.segmenters.base import segmenter_registry
38      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
39
40
41      class FusionSegmenter(TrajectoryHybridSegmenter):
42          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
43
44          family = "fusion"
45          display_label = "Fusion"
46
47          def __init__(self, db: Any, config: Any):
48              super().__init__(db, config)
49              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
50              self._person: Optional[Any] = None
51              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
52              # over the whole trajectory once per candidate window (it depends only on
`points`).
53              self._axis_cache_key: Optional[int] = None
54              self._axis_cache: Optional[tuple] = None
55
56          @property
57          def name(self) -> str:
58              return "fusion_hybrid"
59

```

```

60     @property
61     def description(self) -> str:
62         return (
63             "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
64             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and
the "
65             "AI provider sets semantic boundaries."
66         )
67
68     def _build_runner(
69         self, idx_cfg: IndexingConfig
70     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
71         if isinstance(idx_cfg, dict):
72             idx_cfg = IndexingConfig(**idx_cfg)
73         substrate = idx_cfg.fusion.substrate
74         if substrate == "tracknet":
75             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
76             return TrackNetRunner(cfg), {
77                 "weights_path": cfg.weights_path,
78                 "queue_length": cfg.queue_length,
79                 "fusion_substrate": "tracknet",
80             }
81         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
82         return WasbRunner(wcfg), {
83             "weights_path": wcfg.weights_path,
84             "fusion_substrate": "wasb",
85         }
86
87     def _build_native_runner(
88         self, idx_cfg: IndexingConfig
89     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
90         # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
91         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
92         substrate = idx_cfg.fusion.substrate
93         if substrate != "wasb":
94             raise NotImplementedError(
95                 "detector_impl='native' supports the WASB substrate only ? set "
96                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet)."
97             )
98         from backend.pipeline.detectors.native_wasb_runner import (
99             NativeWasbConfig,
100             NativeWasbRunner,
101         )
102
103         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
104         return NativeWasbRunner(cfg), {
105             "weights_path": cfg.weights_path,
106             "detector_impl": "native",
107             "fusion_substrate": "wasb",
108             "device": cfg.device,
109         }
110
111     def _enrich_candidate_stats(
112         self,
113         cw: Dict[str, Any],
114         points: List[Any],
115         fps: float,
116         frame_width: float,
117         video_path: str,
118         idx_cfg: IndexingConfig,
119     ) -> Dict[str, Any]:
120         stats = super()._enrich_candidate_stats(

```

```

121         cw, points, fps, frame_width, video_path, idx_cfg
122     )
123     # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
124     # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
125     # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
126     gated = rally_gate.gate_windows(
127         points,
128         [(cw["start"], cw["end"])],
129         fps,
130         min_crossings=0,
131         estimate=self._axis_estimate(points),
132     )
133     stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
134
135     fcfg = idx_cfg.fusion
136     if fcfg.cue_flags.get("person", False):
137         stats.update(
138             self._person_features(cw, points, fps, frame_width, video_path, fcfg)
139         )
140     return stats
141
142     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
143     # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
144     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
145     # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
146     # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
147     # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
148     # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
149
150     def _axis_estimate(self, points: List[Any]) -> tuple:
151         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
152         computed once per video, not once per candidate window (it depends only on
points)."""
153         key = id(points)
154         if self._axis_cache_key != key or self._axis_cache is None:
155             self._axis_cache_key = key
156             self._axis_cache = rally_gate.estimate_play_axis(points)
157         est = self._axis_cache
158         assert est is not None
159         return est
160
161     def _person_features(
162         self,
163         cw: Dict[str, Any],
164         points: List[Any],
165         fps: float,
166         frame_width: float,
167         video_path: str,
168         fcfg: "FusionConfig",
169     ) -> Dict[str, float]:
170         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
171         frame range and compute the scale/translation-invariant person features. Lazily
builds
172         ONE PersonDetector. ? real-video verification is owner-side; this wiring is

```

```

mock-tested."""
173         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
174         from backend.pipeline.detectors.person_detector import PersonDetector
175
176         if self._person is None:
177             self._person = PersonDetector(self.config)
178
179         frame_skip = self.config.indexing.yolo_frame_skip
180         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
181         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
182         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
183         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
184         fw = det["frame_width"] or frame_width
185         fh = det["frame_height"]
186         # If we can't trust the frame dims, every normalised feature would silently
collapse
187         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
188         if fw <= 0 or fh <= 0:
189             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
190
191         poly = fcfg.court_polygon
192         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
193         net = (fcfg.net_axis, float(fcfg.net_position))
194         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
195         return ff.compute_person_features(
196             det["frames"], shuttle_in, fw, fh, court_polygon=polygon, net=net
197         )
198
199     def _select_for_handoff(
200         self,
201         windows: List[Dict[str, Any]],
202         window_stats: List[Dict[str, Any]],
203         idx_cfg: IndexingConfig,
204     ) -> List[int]:
205         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
206         (0) keep every window ? identical to the substrate. Persisted candidates are
207         untouched (full superset).
208
209         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
210         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
211         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
212         set rather than emit a video with zero rallies. This makes Gate A/B strictly
213         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
214         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
215         below it always wins on an otherwise-empty video. Observed on real footage
216         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
217         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
218         fcfg = idx_cfg.fusion
219         min_crossings = int(fcfg.min_crossings)
220         min_players = int(fcfg.min_players)
221         person_on = bool(fcfg.cue_flags.get("person", False))
222
223         keep: List[int] = []
224         for i, st in enumerate(window_stats):
225             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:

```

```

226         continue
227     if (
228         person_on
229         and min_players > 0
230         and st.get("players_in_court", 0.0) < min_players
231     ):
232         continue
233     keep.append(i)
234
235     if not keep and window_stats:
236         # Gating emptied a non-empty video ? fall back to the ungated set.
237         return list(range(len(window_stats)))
238     return keep
239
240
241     segmenter_registry.register(FusionSegmenter)
242

```

13. assistant (2026-07-06T18:01:17.268Z)

Fusion inherits `process\_video` from `TrajectoryHybridSegmenter` and only overrides seams, so F1/F3 apply correctly. Now let me look at the two remaining callers (`main.py:1552` and `orchestration.py:795`) to confirm they branch on `isinstance(result, Failure)`, and check the gemini segmenter which the changes claim to mirror.

14. user (2026-07-06T18:01:19.283Z)

```

1540         return
1541
1542         # Index
1543         segmenter_cls = segmenter_registry.get(seg_name)
1544         if not segmenter_cls:
1545             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546             return
1547
1548         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549         segmenter_actual = seg_name
1550         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551         segmenter = segmenter_cls(db_instance, global_config)
1552         result = segmenter.process_video(
1553             video_id, local_video, max_frames=args.max_frames
1554         )
1555         segmentation_ran = True
1556
1557         if isinstance(result, Failure):
1558             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559             print("[FAIL] Indexing halted for video.")
1560             segments = []
1561         else:
1562             segments = result.value if hasattr(result, "value") else result
1563             print(
1564                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565             )
1566             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568         finally:
1569             # Always emit the per-video observability report. Never raises.
1570             paths = emit_indexer_report(
1571                 db=db_instance,
1572                 video_id=video_id,
1573                 # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                 # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source

```

```

URI is kept
1575         # in DB provenance via db.add_video.
1576         video_path=local_video,
1577         config=_report_config_for_input(global_config, args.video_path),
1578         val_results=val_results,
1579         val_context=val_context,
1580         segmenter_requested=args.segmenter,
1581         segmenter_actual=segmenter_actual,
1582         was_reingest=was_reingest,
1583         segmentation_ran=segmentation_ran,
1584     )
1585     if paths:
1586         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1589     finally:
1590         storage.cleanup_acquired_local_input(
1591             args.video_path, local_video, storage_cfg
1592         )
1593     return
1594
1595     # Start Local Server Mode
1596     if args.app or args.server or not args.video_path:
1597         if args.app:
1598             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1600             bind_host = "127.0.0.1"
1601             port = _resolve_app_port(bind_host, args.port)
1602             if port != args.port:
1603                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1604             url = f"http://{bind_host}:{port}/"
1605             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1606             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1607             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1608             print(
1609                 f"[APP] Starting KhelSutra ? opening {url} in your browser "

```

15. user (2026-07-06T18:01:19.926Z)

```

780         remote = _maybe_dispatch_remote(
781             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
782         )
783         if remote is not None:
784             response, segmentation_ran = (
785                 remote # `ran` = did the cloud job actually execute (report)
786             )
787             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
788         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
789         compute_actual = "in_process"
790         segmenter = segmenter_cls(db, config)
791         try:
792             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
793             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
794             with _LOCAL_COMPUTE_LANE:
795                 result = segmenter.process_video(
796                     video_id, local_video, req.player_pool, req.max_frames

```

```

797         )
798     except Exception:
799         # A raising segmenter must not leave half-written rows: restore the pre-run
800         # state (same destroy-after-confirm contract as the Failure branch), then
801         let
802             # the job worker record the failure.
803             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
804             raise
805     segmentation_ran = True
806
807     if isinstance(result, Failure):
808         logger.error(f"[API] Segmenter failed: {result.message}")
809         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
810         response = {
811             "video_id": video_id,
812             "status": "failed",
813             "message": f"Segmenter '{seg_name}' failed: {result.message}",
814             "results": [r.model_dump() for r in val_results],
815             "play_segments": [],
816         }
817         return response
818
819     segments = result.value if hasattr(result, "value") else result
820     notify("finalizing")
821     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
822
823     response = {
824         "video_id": video_id,
825         "status": "success",
826         "message": f"Successfully indexed {len(segments)} play segments using
827         '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
828         "results": [r.model_dump() for r in val_results],
829         "play_segments": segments,
830     }
831     return response
832
833     finally:
834         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
835         already set
836         # response["compute"] (path=cloud_run + the Cloud Run execution); for every
837         other path stamp
838         # the actual path here so the job result PROVES which backend ran (in_process /
839         not_run) and
840         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
841         then visible.
842         if isinstance(response, dict):
843             response.setdefault(
844                 "compute",
845                 {
846                     "path": compute_actual or "not_run",
847                     "requested": getattr(req, "compute", None),
848                 },
849             )
850         prov = response.get("compute", {}) if isinstance(response, dict) else {}
851         logger.info(
852             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
853             video_id,
854             getattr(req, "compute", None),
855             prov.get("path"),
856             prov.get("dispatched"),
857         )

```

16. assistant (2026-07-06T18:01:23.826Z)

Both callers correctly branch on `isinstance(result, Failure)`. Now let me verify there are no remaining `is None` checks on the `\_resolve\_provider` result anywhere, and check the gemini

segmenter (which these changes claim to mirror) plus how the empty `all\_candidate\_windows` path in `\_run\_ai\_handoff` behaves when yolo produces zero windows.

17. user (2026-07-06T18:01:26.033Z)

```
backend\pipeline\segmenters\gemini.py:84: def _resolve_provider(self, model_name: str):
backend\pipeline\segmenters\gemini.py:636: resolved = self._resolve_provider(model_name)
backend\pipeline\segmenters\yolo_hybrid.py:66: def _resolve_provider(self, video_path: str):
backend\pipeline\segmenters\yolo_hybrid.py:500: provider =
self._resolve_provider(video_path)
```

18. user (2026-07-06T18:01:27.014Z)

```
600         "detector": model_name,
601     }
602
603     segment_id = self.db.add_play_segment(
604         video_id, start, end, None, None, metadata
605     )
606
607     if segment_id:
608         segments_data.append(
609             {
610                 "id": segment_id,
611                 "start_time": start,
612                 "end_time": end,
613                 "duration": dur,
614                 "metadata": metadata,
615             }
616         )
617
618     return segments_data
619
620 def process_video(
621     self,
622     video_id: str,
623     video_path: str,
624     player_pool: List[str] = None,
625     max_frames: int = None,
626 ) -> List[Dict[str, Any]]:
627     """
628     Processes a video to detect play segments using Google Gemini Multimodal Video
629     API,
630     populates SQLite, and indexes detailed events.
631     """
632     model_name = self.config.ai_model
633
634     # Resolve sport profile + provider. An unrunnable config (unknown sport /
635     # missing key /
636     # unknown provider) comes back as a Failure ? propagate it so the job is marked
637     # FAILED
638     # with a reason, not a silent 0-rally success (Finding B).
639     resolved = self._resolve_provider(model_name)
640     if isinstance(resolved, Failure):
641         return resolved
642     sport_profile, provider = resolved
643
644     # Optionally trim the video first (debug feature); cleanup removes any temp
645     # file.
646     video_to_upload, cleanup_temp_files = self._maybe_trim_video(
647         video_id, video_path
648     )
649
650     # 1. Determine actual video duration
651     video_duration = get_video_duration(video_to_upload)
```

```

648         if video_duration > 0:
649             logger.info(f"Detected video duration: {video_duration:.1f}s")
650         else:
651             logger.warning(
652                 "Could not detect video duration. Prompt will not include duration
context."
653             )
654
655         # 2. Decide whether to use chunked processing
656         idx_cfg = self.config.indexing
657         chunk_duration = idx_cfg.chunk_duration
658         chunk_overlap = idx_cfg.chunk_overlap
659         use_chunking = video_duration > 0 and video_duration > chunk_duration
660         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
661         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
662         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
663         ai_scale_width = int(idx_cfg.ai_chunk_scale_width or 0)
664
665         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
666         # video keeps its own dir, so cleanup here never removes another run's chunks.
667         chunks_dir = _chunk_workdir(video_id, output_base(self.config))
668
669         if use_chunking:
670             ai_segments = self._process_chunked(
671                 video_id,
672                 video_to_upload,
673                 sport_profile,
674                 provider,
675                 video_duration,
676                 chunk_duration,
677                 chunk_overlap,
678                 ai_scale_width,
679                 chunks_dir,
680                 cleanup_temp_files,
681             )
682         else:
683             ai_segments = self._process_single(
684                 video_id,
685                 video_to_upload,
686                 sport_profile,
687                 provider,
688                 video_duration,
689                 cleanup_temp_files,
690             )
691
692         if not ai_segments:
693             return []
694
695         # Populate SQLite DB with parsed play segments
696         validated = self._validate_and_dedupe(ai_segments, video_duration)
697
698         # 9. Populate SQLite DB with validated play segments
699         segments_data = self._populate_db(video_id, validated, model_name)
700
701         return segments_data
702
703
704     # Register the segmenter
705     segmenter_registry.register(GeminiPlaySegmenter)
706

```

19. assistant (2026-07-06T18:01:38.445Z)

Confirmed: no remaining `provider is None` checks. Both `\_resolve\_provider` callers

(yolo_hybrid:500, gemini:636) now branch on `isinstance(..., Failure)`.

Now I need to carefully check a key edge case in the F3 change. In `yolo\_hybrid.\_run\_ai\_handoff`, when `all\_candidate\_windows` is empty, the loop runs zero times, `handoff\_failed=0`, and the guard `all\_candidate\_windows and handoff\_failed == len(all\_candidate\_windows)` is `False` (empty list is falsy), so it returns `gemini\_segments` (empty `[]`). That is correct ? zero windows is a legit empty. Same for trajectory_hybrid where the empty-windows case returns early at line 380-381 before reaching handoff.

Let me now scrutinize a subtle concern: the **`extract\_video\_chunk`** failure counting as "errored" in the all-errored guard. Consider a partial run where SOME windows extract-fail (transient) but others succeed with real rallies ? that stays a Success (correct). But consider the case where clip extraction fails on EVERY window while the provider itself is healthy. Let me check what `extract\_video\_chunk` failure means and whether an all-extraction-failed run should truly be recoverable. Let me also examine the tracknet/wasb test helpers to verify the tests exercise what they claim.

20. user (2026-07-06T18:01:40.479Z)

```
1      """Tests for the TrackNetHybridSegmenter pipeline. The WSL/TensorFlow runner is
2      mocked out (TrackNetRunner), so these run anywhere ? they exercise Phase 1-4 glue:
3      healthcheck gating, candidate persistence, AI handoff, and DB population/linking."""
4
5      from unittest.mock import MagicMock, patch
6
7      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
8      from backend.results import Failure
9
10
11     def _window(start=1.0, end=4.0):
12         return {
13             "start": start,
14             "end": end,
15             "active_frame_count": 30,
16             "peak_velocity": 0.4,
17             "mean_velocity": 0.2,
18             "track_id_count": 1,
19             "chunk_index": 0,
20         }
21
22
23     def _patched(fn):
24         """Stack the patches every pipeline test needs. fps/width probing is stubbed
25         so cv2 internals don't need mocking (covered separately).
26
27         Patches are applied innermost-first, and mock.patch injects the innermost
28         patch as the *first* positional arg ? so this order matches the signature
29         (mock_runner_cls, mock_probe, mock_dur, mock_extract, mock_provider_registry,
30         mock_env)."""
31         fn = patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
32         fn = patch.object(
33             TrackNetHybridSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)
34         )(fn)
35         # Shared pipeline collaborators live in the trajectory_hybrid base module.
36         fn = patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration")(fn)
37         fn = patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk")(fn)
38         fn = patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry")(fn)
39         fn = patch("os.environ.get")(fn)
40         return fn
41
42
43     def _make_runner(windows=None, csv="out.csv", healthy=True):
44         runner = MagicMock()
45         runner.healthcheck.return_value = (
46             healthy,
```

```

47         "TF 2.17 GPUs 1" if healthy else "no GPU",
48     )
49     runner.run_predict.return_value = csv
50     runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
51     runner.trajectory_to_action_windows.return_value = (
52         windows if windows is not None else [_window()]
53     )
54     return runner
55
56
57 @patched
58 def test_pipeline_happy_path(
59     mock_runner_cls,
60     mock_probe,
61     mock_dur,
62     mock_extract,
63     mock_provider_registry,
64     mock_env,
65 ):
66     mock_env.return_value = "dummy_key"
67     mock_dur.return_value = 30.0
68     mock_extract.return_value = True
69     mock_runner_cls.return_value = _make_runner()
70
71     provider = MagicMock()
72     provider.analyze_video.return_value = (
73         [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}],
74         {},
75     )
76     mock_provider_registry.get.return_value = provider
77
78     db = MagicMock()
79     db.add_candidate_segment.return_value = 55
80     db.add_play_segment.return_value = 200
81
82     seg = TrackNetHybridSegmenter(db, {"indexing": {}})
83     res = seg.process_video("v1", "clip.mp4")
84
85     assert len(res) == 1
86     assert res[0]["id"] == 200
87     # Candidate persisted under the tracknet detector name...
88     _, kwargs = db.add_candidate_segment.call_args
89     assert kwargs["detector"] == "tracknet_hybrid"
90     # ...and the confirmed segment linked back to it.
91     db.link_candidate_to_segment.assert_called_once_with(55, 200)
92
93
94 @patched
95 def test_pipeline_no_api_key_is_failure(

```

21. user (2026-07-06T18:01:41.269Z)

```

1     """Equivalence contract for the trajectory-hybrid twins.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4     shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5     These tests run BOTH through the same scenarios and assert identical P3/P4
6     behavior, so any future divergence has to be introduced deliberately, in the
7     base, for both at once.
8     """
9
10    from unittest.mock import MagicMock, patch
11
12    import pytest
13

```

```

14 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
15 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
17 from backend.results import Failure
18
19 TWINS = [
20     (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
21     (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
22 ]
23
24
25 def _window(start=1.0, end=4.0):
26     return {
27         "start": start,
28         "end": end,
29         "active_frame_count": 30,
30         "peak_velocity": 0.4,
31         "mean_velocity": 0.2,
32         "track_id_count": 1,
33         "chunk_index": 0,
34     }
35
36
37 def _runner(windows=None):
38     runner = MagicMock()
39     runner.healthcheck.return_value = (True, "env ok")
40     runner.run_predict.return_value = "out.csv"
41     runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
42     runner.trajectory_to_action_windows.return_value = (
43         windows if windows is not None else [_window()]
44     )
45     return runner
46
47
48 def _run(cls, provider_segments, db=None):
49     """Run one segmenter class through the standard mocked pipeline."""
50     provider = MagicMock()
51     provider.analyze_video.return_value = (provider_segments, {})
52     db = db or MagicMock()
53     db.add_candidate_segment.return_value = 55
54     db.add_play_segment.return_value = 200
55
56     with patch.object(
57         cls, "_build_runner", return_value=(_runner(), {"weights_path": "w"}),
58     ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
59         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
60         return_value=30.0,
61     ), patch(
62         "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
63         return_value=True,
64     ), patch(
65         "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
66     ) as preg, patch("os.environ.get", return_value="dummy_key"):
67         preg.get.return_value = provider
68         seg = cls(db, {"indexing": {}})
69         return db, seg.process_video("v1", "clip.mp4")
70
71
72 @pytest.mark.parametrize("cls,family,detector", TWINS)
73 def test_detector_identity_flows_into_db_rows(cls, family, detector):
74     db, res = _run(
75         cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}]
76     )
77     assert len(res) == 1
78     _, kwargs = db.add_candidate_segment.call_args

```

```

79     assert kwargs["detector"] == detector
80     assert kwargs["detection_params"]["detector"] == detector
81     # P4 metadata tags the family + model.
82     assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
83     db.link_candidate_to_segment.assert_called_once_with(55, 200)
84
85
86 @pytest.mark.parametrize("cls, family, detector", TWINS)
87 @pytest.mark.parametrize(
88     "exchange, expected",
89     [
90         (4, 4), # provider-reported count wins
91         ("7", 7), # numeric strings coerce
92         (None, 3), # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
93         ("garbage", 3), # unparseable -> same estimate
94     ],
95 )
96 def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
97     """The historical drift point: wasb_hybrid relied on int(None) raising into
98     the fallback while tracknet_hybrid short-circuited. One implementation now."""
99     seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
100     _, res = _run(cls, [seg_dict])
101     assert res[0]["metadata"]["shot_count"] == expected
102
103
104 @pytest.mark.parametrize("cls, family, detector", TWINS)
105 def test_provider_offsets_applied_identically(cls, family, detector):
106     # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
107     # provider-relative 0.5 stays 0.5 absolute here.
108     _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
109     assert res[0]["start_time"] == 0.5
110     assert res[0]["end_time"] == 3.0
111
112
113 @pytest.mark.parametrize("cls, family, detector", TWINS)
114 def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
115     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
116     loudly, not
117     silently treated as 'no rallies'. When it errors on the ONLY candidate window the
118     whole run has
119     zero verdict, so process_video now returns a Failure (was a bare [] that looked like
120     a 0-rally
121     match) ? the worker marks it failed/retryable instead of a silent empty
122     'success'."""
123     import logging
124
125     provider = MagicMock()
126     provider.analyze_video.return_value = (
127         [],
128         {"error": "generate failed after retries: 503", "generate_failed": True},
129     )
130     db = MagicMock()
131     db.add_candidate_segment.return_value = 55
132     with patch.object(
133         cls, "_build_runner", return_value=_runner(), {"weights_path": "w"})
134     ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
135         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
136         return_value=30.0,
137     ), patch(
138         "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
139         return_value=True,
140     ), patch(
141         "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
142     ) as preg, patch("os.environ.get", return_value="dummy_key"):
143         preg.get.return_value = provider

```

```

140         seg = cls(db, {"indexing": {}})
141         with caplog.at_level(
142             logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
143         ):
144             res = seg.process_video("v1", "clip.mp4")
145         assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored ->
no verdict
146         assert any(
147             "provider error" in r.message and "generate failed" in r.message
148             for r in caplog.records
149         ), [r.message for r in caplog.records]
150
151
152     @pytest.mark.parametrize("cls,family,detector", TWINS)
153     def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
154         """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
155         no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
156         handed off; the first returns a rally, the second errors -> a Success with the first
rally."""
157         provider = MagicMock()
158         provider.analyze_video.side_effect = [
159             ( [{"start_time": 0.5, "end_time": 2.0}], {} ), # window 0: a real verdict
160             ( [], {"error": "generate failed after retries: 503"} ), # window 1: errored

```

22. assistant (2026-07-06T18:01:46.728Z)

The tests are well-constructed. Now let me carefully examine one subtle F3 edge case in the ``test_partial_handoff_error_is_a_success_not_a_failure`` test. The provider uses ``side_effect`` (ordered), but the handoff runs in a ``ThreadPoolExecutor`` with ``ai_max_workers``. If ``ai_max_workers` > 1`, the two windows could be submitted concurrently and ``side_effect`` would be consumed in call-order, which may not match window order. But since ``window_results`` is collected in future submission order and results are keyed to their window, the aggregation is order-safe. Let me check the default ``ai_max_workers`` to see if concurrency could make the test flaky, and verify the ``_run_ai_handoff`` empty-windows scenario for yolo more carefully.

Let me check the default config for ``ai_max_workers``:

23. user (2026-07-06T18:01:48.471Z)

```
backend\config\models.py:254:    ai_max_workers: int = 5
```

24. assistant (2026-07-06T18:01:59.139Z)

``ai_max_workers=5``, so with 2 windows both are submitted concurrently. The ``provider.analyze_video.side_effect`` is a list consumed by call arrival order, not window index. This is a potential test-flakiness concern. Let me analyze precisely.

In ``test_partial_handoff_error_is_a_success_not_a_failure``: window 0 should return the rally ``[{start:0.5...}]``, window 1 should error. Because ``side_effect`` is consumed in `**call order**` (whichever thread reaches ``analyze_video`` first), and both threads run concurrently, there's a race: if window 1's thread calls ``analyze_video`` first, it gets the rally result and window 0 gets the error. The test then asserts ``res[0]["start_time"] == 0.5`` and ``len(res) == 1``.

Let me trace whether the coordinate mapping would break this assertion. In the trajectory test, ``two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]`` with ``handoff_padding`` default. Let me check the default padding, because the segment's ``start_time`` gets offset by ``w_start``.

25. user (2026-07-06T18:02:00.475Z)

```
253:    ai_handoff_padding: float = 2.0
```

26. assistant (2026-07-06T18:02:13.967Z)

This is a real test-flakiness concern. Let me trace it precisely for both the trajectory and yolo tests.

```
**Trajectory test** (`test_partial_handoff_error_is_a_success_not_a_failure`):  
- `two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]`, `handoff_padding = 2.0`  
- Window 0: `w_start = max(0, 1.0 - 2.0) = 0.0`. If it gets the rally result `{start_time: 0.5}`, the offset segment is `0.5 + 0.0 = 0.5`. ? assertion passes.  
- Window 1: `w_start = max(0, 10.0 - 2.0) = 8.0`. If window 1 gets the rally result (due to race), the offset segment is `0.5 + 8.0 = 8.5`, not `0.5`. ? assertion `res[0]["start_time"] == 0.5` **fails**.
```

Since `ai\_max\_workers=5`, both windows are submitted to the executor concurrently. `side\_effect` is a list, and `MagicMock` pops from it in the order calls arrive ? which is non-deterministic under concurrency. Whichever thread reaches `provider.analyze\_video()` first gets `[{start:0.5}], {}`.

So the test asserts a result that depends on thread scheduling. This is a genuine flakiness bug in the new tests. Let me verify there's no serialization I'm missing (e.g., MagicMock side_effect being thread-safe/ordered, or the executor somehow serializing). MagicMock call handling isn't strictly thread-safe, but the ordering issue is the real problem: `side\_effect` list consumption follows call order, not submission order.

Let me double-check whether `ThreadPoolExecutor` with `max\_workers=5` and 2 submitted tasks actually runs them concurrently (it does ? worker threads are spawned lazily up to max_workers as tasks are submitted). Let me confirm the test doesn't override `ai\_max\_workers`. In the trajectory test, config is `{ "indexing": {} }` so it uses default `ai\_max\_workers=5`. Same for the yolo test.

Let me verify the yolo test's window offsets too:

27. user (2026-07-06T18:02:15.017Z)

```
153         assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored ->  
no verdict  
154         assert any(  
155             "provider error" in r.message and "generate failed" in r.message  
156             for r in caplog.records  
157         ), [r.message for r in caplog.records]  
158  
159  
160         @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")  
161         @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")  
162         @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")  
163         @patch("os.environ.get")  
164         def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(  
165             mock_env_get, mock_provider_registry, mock_extract, mock_get_dur  
166         ):  
167             """The boundary of F3: SOME windows errored but at least one RAN, so the run is a  
Success (the  
168             first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows  
directly."""  
169             mock_env_get.return_value = "dummy_api_key"  
170             mock_get_dur.return_value = 30.0  
171             mock_extract.return_value = True  
172  
173             mock_provider = MagicMock()  
174             mock_provider.analyze_video.side_effect = [  
175                 ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict  
176                 ([], {"error": "generate failed after retries: 503"}), # window 1: errored  
177             ]  
178             mock_provider_registry.get.return_value = mock_provider  
179  
180             db_mock = MagicMock()  
181             db_mock.add_play_segment.return_value = "yolo_segment_1"  
182             two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
```

```

183
184     segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
185     segmenter.person.model = MagicMock() # short-circuit lazy model load
186     with patch.object(
187         HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
188     ), patch.object(
189         HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
190     ):
191         res = segmenter.process_video("v1", "dummy.mp4")
192         assert not isinstance(res, Failure)
193         assert len(res) == 1 and res[0]["start_time"] == 0.5
194
195
196     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
197     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")

```

28. assistant (2026-07-06T18:02:26.275Z)

Same pattern in the yolo test: ``two_windows = [{start:1.0,end:4.0}, {start:10.0,end:13.0}]``, ``handoff_padding=2.0``, ``ai_max_workers=5`` (default). Window 0 ? ``w_start=0.0``; window 1 ? ``w_start=8.0``. ``side_effect`` consumed in call-arrival order under concurrency. If window 1's thread calls ``analyze_video`` first, it gets the rally and the offset becomes ``0.5 + 8.0 = 8.5``, breaking ``res[0]["start_time"] == 0.5``.

This is a real concurrency-ordering flakiness bug in the two new tests. However, before flagging it as a defect, I should assess the actual probability and whether existing passing tests (``test_provider_offsets_applied_identically``) already rely on the same executor with a single window ? they do, but with a single window there's no ordering issue.

Let me check whether there's any existing precedent test in the suite that hands off multiple windows with ``side_effect`` and passes reliably, to calibrate whether the executor's behavior with a submission-then-result pattern happens to serialize in practice. Let me look at the happy-path multi-window tests if any exist.

29. user (2026-07-06T18:02:28.535Z)

```

31:     cap_mock.get.side_effect = mock_get_prop
32:     cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
54:         side_effect=_make_detections(num_frames, step)
72:     mock_provider.analyze_video.return_value = (
125:     mock_provider.analyze_video.return_value = (
174:     mock_provider.analyze_video.side_effect = [
182:         two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
187:         HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
234:     mock_provider.analyze_video.return_value = (
318:     mock_provider.analyze_video.return_value = (
371:     mock_provider.analyze_video.return_value = (
423:     mock_provider.analyze_video.return_value = (

```

30. assistant (2026-07-06T18:02:38.677Z)

The only multi-window ``side_effect`` test is the new one at line 174. All other tests use ``return_value`` (order-independent). So there's no existing precedent proving the executor serializes.

Let me confirm my analysis about ``ThreadPoolExecutor`` concurrency with 2 tasks and ``max_workers=5``. The executor spawns a new worker thread for each submitted task until reaching `max_workers`, if there are pending work items. With ``submit`` called twice in a list comprehension, two work items are queued; the executor spawns up to 2 threads that both pull work concurrently. So both ``analyze_video`` calls can genuinely run in parallel, and ``side_effect`` (a list) is consumed by whichever call's ``__call__`` executes first.

This is a legitimate flakiness defect. Let me verify the severity by confirming ``MagicMock.side_effect`` list consumption is call-order based (it is ? ``side_effect`` iterables

advance on each call regardless of args). So the test result depends on which thread calls first.

Now let me also verify one more thing the review asks about: whether the `\_run\_ai\_handoff` in yolo, when returning a `Failure`, is properly checked before `\_populate\_db`. I already confirmed at line 588-589 it checks `isinstance(handoff, Failure)` before line 592 `\_populate\_db(video\_id, handoff, ...)` . Good.

Let me now check the remaining callers I haven't fully vetted for regressions ? the control plane consumers mentioned: `orchestration.cached\_process\_result`, `job\_worker.\_resume\_one\_remote\_job`, `compute.cloud\_run`. Let me grep for how they consume report.json status.

31. user (2026-07-06T18:02:44.580Z)

```
Found 12 files
backend\worker\__main__.py
backend\ui\assets\index-503c-FCB.js
backend\orchestration.py
backend\main.py
backend\infrastructure\database_jobs.py
backend\api\job_worker.py
backend\compute\cloud_run.py
backend\compute\base.py
backend\api\jobs.py
backend\infrastructure\database_media.py
backend\providers\gemini.py
backend\api\static\app.js
```

32. assistant (2026-07-06T18:02:47.042Z)

Let me check the control-plane consumers of report.json status to confirm the new `status="failed"` reports don't regress them (they should treat non-"success" as retry/no-usable-result).

33. user (2026-07-06T18:02:49.179Z)

```
8:      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
15:      ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
236:      report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
```

34. user (2026-07-06T18:02:50.819Z)

```
3:The full hash -> validate -> segment -> report pipeline (`_execute_processing`), the
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
80:def _report_config_for_input(config: Any, raw_input: str) -> Any:
81:    """Return report config that keeps remote-input reports out of disposable scratch."""
100:    report = dict(cfg.get("report") or {})
101:    if report.get("output_dir"):
103:        report["output_dir"] = os.path.join(output_base(config), "reports")
104:        cfg["report"] = report
217:        "status": "success",
233:    with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
235:        uri, os.path.join(td, ref.name or "report.json"), config=storage_config
255:def probe_process_cache(db, cfg, video_id: str) -> bool:
261:    errors report False (? the normal dispatch path with its guards)."""
266:        and existing.get("status") == "processed"
276:        report_uri = f"{output_root}/outputs/{video_id}/report.json"
277:        if not storage.exists(report_uri, config=sc):
279:            report = _read_bucket_json(report_uri, sc)
280:            return bool(report and report.get("status") == "success")
288:def cached_process_result(
292:    the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
```

```

306:         if existing and existing.get("status") == "processed":
322:             report_uri = f"{outputs_uri}/report.json"
323:             if not storage.exists(report_uri, config=sc):
325:                 report = _read_bucket_json(report_uri, sc)
326:                 if not report or report.get("status") != "success":
329:                     # Read-path re-hydrate (#485): a wiped DB has no source record. The report
330:                     # carries the worker's --video-uri (P3); older reports fall back to the
334:                     video_path = str(report.get("video_uri") or "") or _first_object_under(
378:                     """"#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
382:                     bucket; every failure degrades to False (the read just reports what the DB has).""")
386:                     if not probe_process_cache(db, cfg, vid):
388:                         return cached_process_result(db, cfg, vid, "") is not None
407:                 actually EXECUTED (``False`` for a pre-dispatch config rejection, so the observability
report
424:                     "status": "failed",
624:                     "status": "success",
637:                 """"The full hash ? validate ? segment ? report pipeline for one video.
670:                 # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
709:                     and existing_video.get("status") == "processed"
726:                 # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
734:                     # status; it no longer destroys the video's prior good segments.
743:                     "status": "failed",
762:                     "status": "failed",
785:                     remote # `ran` = did the cloud job actually execute (report)
811:                     "status": "failed",
824:                     "status": "success",
845:                     "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
850:                     (response or {}).get("status") if isinstance(response, dict) else None,
852:                 # Always emit the per-video observability report (next to the video, or to
853:                 # report.output_dir). Never raises. Surface the paths in the response. (The compute-
path
854:                 # proof lives on the job result + the [COMPUTE] log, not the report ? obsreport's
record_run
856:                 paths = _main.emit_indexer_report(
859:                 # Resolved LOCAL path, not the raw URI: obsreport writes the report next to the
864:                 config=_report_config_for_input(config, req.video_path),
873:                 response["reports"] = paths
901:                 """"A submit-time-validatable compile problem ? carries the HTTP status + detail so the
API path
904:                 def __init__(self, status_code: int, detail: str):
906:                     self.status_code = status_code
993:                 and return the {status, filepath, download_url} result the SPA polls for. Returns a
994:                 ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1010:                 return {"status": "failed", "message": e.detail, "video_id": video_id}

```

[Showing results with pagination = limit: 60]

35. assistant (2026-07-06T18:02:55.485Z)

The control-plane consumers are correct: `probe\_process\_cache` (line 280) and `cached\_process\_result` (line 326) both require `report.get("status") == "success"`, so a new `status="failed"` report correctly reads as "no usable result" ? the normal dispatch path fires. No regression ? this is exactly the intended behavior. The `cloud\_run.py` reads report.json into a `ComputeResult` at line 236; let me confirm that path handles the failed status without regressing (it uses the done-marker's returncode, not the report status, for success/failure determination per the module docstring at line 8).

Let me verify the cloud_run ComputeResult construction to ensure a failed report doesn't get misinterpreted as success.

36. user (2026-07-06T18:02:57.286Z)

```

210         )
211
212     try:
213         marker = poll_until(
214             lambda: self._read_marker(done_uri),
215             self._policy,
216             label="cloud-run-infer",
217             logger=LOG,
218             # Live heartbeat (#482): each still-waiting tick reaches the caller so
219             # job.progress moves during the whole GPU run instead of freezing.
220             on_tick=on_wait,
221         )
222     except PolicyTimeout as timeout_exc:
223         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
224         video_id = str(marker.get("video_id", ""))
225         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
226         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
227         rc = int(marker.get("returncode", 1))
228         if not video_id:
229             LOG.warning(
230                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
231                 "video_id) ? treating as failure",
232                 self._job_name,
233             )
234             rc = rc or 1
235         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
236         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
237         LOG.info(
238             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
239         )
240         return ComputeResult(
241             returncode=rc,
242             outputs_uri=outputs_uri,
243             video_id=video_id,
244             report=report,
245             execution=str(execution or ""),
246         )
247
248     def _handle_poll_timeout(
249         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
250     ) -> dict:
251         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
252
253         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
job's own
254         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
255         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
256         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
257         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
258         execution name so the operator can verify the state in the GCP console. A cancel
failure
259         NEVER masks the original timeout."""
260         try:
261             late = self._read_marker(done_uri)
262             except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
263                 LOG.warning(
264                     "cloud-run: final post-timeout marker check failed ? treating as
absent",

```

```

265         exc_info=True,
266     )
267     late = None
268     if late is not None:
269         LOG.warning(
270             "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
271             "? rescuing the completed result",
272             self._job_name,
273             execution,
274         )
275     return late
276     try:
277         self._client.cancel_execution(execution)
278     except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
279         cancel_note = f"best-effort cancel FAILED ({cancel_exc})"

```

37. assistant (2026-07-06T18:03:02.022Z)

Confirmed: `cloud\_run` derives success/failure from the done-marker's `returncode` (line 227), not the report status. A `\_fail(3)` writes a marker with `returncode=3`, so `rc=3` correctly propagates as a failure. The report is just attached as metadata. No regression.

I have now fully traced the control plane. Let me do a final check on the `job\_worker.\_resume\_one\_remote\_job` path to confirm the newly-failed report doesn't regress restart-recovery, and confirm there are no other tests in the repo that assert these paths return `[]`.

38. user (2026-07-06T18:03:05.213Z)

```

371:         target=_resume_one_remote_job,
386: def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
477:     status = str(report.get("status") or "unreadable")

```

39. user (2026-07-06T18:03:06.671Z)

No matches found

40. user (2026-07-06T18:03:09.766Z)

```

460     )
461     return
462     payload = orchestration.cached_process_result(
463         db, config, video_id, str(job.get("video_path") or "")
464     )
465     if payload is not None:
466         db.mark_job_done(job_id, payload)
467         logger.warning(
468             "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469             job_id,
470             video_id,
471         )
472     return
473     # The report exists but did not realize into a usable result ? surface its
verdict.
474     status = "unreadable"
475     try:
476         report = orchestration._read_bucket_json(report_uri, sc) or {}
477         status = str(report.get("status") or "unreadable")
478     except Exception: # noqa: BLE001 ? the failure message is best-effort
479         pass
480     db.mark_job_failed(
481         job_id,

```

```

482         f"cloud worker finished without a usable result after a control-plane
restart "
483         f"(report status: {status}); re-submit the video",
484     )
485     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
486         logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487         try:
488             db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489         except Exception: # noqa: BLE001
490             pass
491

```

41. assistant (2026-07-06T18:03:18.954Z)

Confirmed: `\_resume\_one\_remote\_job` uses `cached\_process\_result` which requires `status == "success"`. For a `status="failed"` report, `cached\_process\_result` returns `None`, so line 473-484 fires and marks the job failed with the report's status verdict. This is correct ? no regression, and in fact an improvement (previously a silent-success 0-segment report would have resolved as a usable empty result).

Let me now confirm the `\_resume\_one\_remote\_job` restart-recovery poll's precondition ? the review note says "the restart-recovery poll waits for report.json to EXIST". Let me verify that a `\_fail` path now writing report.json satisfies that poll so a genuinely-failed remote job doesn't hang forever waiting for a report that (before this change) was never written. Let me look above line 460 for the report-exists wait.

42. user (2026-07-06T18:03:19.695Z)

```

420         db.mark_job_failed(job_id, "interrupted by server restart")
421         return
422     sc = orchestration._storage_settings(config)
423     output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424     if not sc or not output_root:
425         db.mark_job_failed(
426             job_id,
427             "interrupted by server restart (no output bucket configured to recover
from)",
428         )
429         return
430     report_uri = f"{output_root}/outputs/{video_id}/report.json"
431     max_wait = float(
432         getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433         or 3900.0
434     )
435     policy = ExecutionPolicy(
436         poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437         op_timeout_sec=120.0,
438         max_wait_sec=max_wait,
439     )
440     db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441     try:
442         poll_until(
443             lambda: True if storage.exists(report_uri, config=sc) else None,
444             policy,
445             label=f"resume-{job_id[:8]}",
446             logger=logger,
447             # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448             on_tick=lambda elapsed: db.set_job_progress(
449                 job_id,
450                 "recovering: waiting for the in-flight cloud run ? "
451                 f"{int(elapsed) // 60} min",
452             ),

```

```

453         )
454     except PolicyTimeout:
455         db.mark_job_failed(
456             job_id,
457             "control plane restarted mid-run and the cloud worker's result never "
458             "appeared within the poll budget; re-submit the video (a finished result "
459             "re-hydrates instantly)",
460         )
461     return

```

43. assistant (2026-07-06T18:03:33.005Z)

This confirms the design: the restart-recovery poll (line 442-443) waits for `report.json` to EXIST. Before this change, a `\_fail(3)` path did NOT write report.json (the old `\_fail` only wrote the done-marker), so a control-plane restart during a genuinely-failed remote job would hang until the poll budget expired (~65 min) before failing. After the change, `\_fail` writes report.json, so the poll terminates FAST and correctly. This is a real improvement and confirms the completeness of the fix ? no regression.

I have now comprehensively verified all four lenses:

1. **F1/F2 remaining bare-`[]` paths** ? enumerated and classified all: trajectory_hybrid returns bare `[]` at line 381 (empty candidate windows ? legit) and line 535 (segments_data after DB population ? legit); yolo_hybrid returns bare list at line 592 (`\_populate\_db` output ? legit) and `\_run\_ai\_handoff` line 431 (`gemini\_segments` ? legit when at least one window ran). All config/env/input failure paths are now `Failure`. No missed sibling failure path, no wrongly-converted legit-empty path.
2. **F2 caller interaction** ? `process\_video` checks `isinstance(handoff, Failure)` (line 588) before `\_populate\_db` (line 592); no remaining `is None` checks anywhere.
3. **Fusion inheritance** ? inherits `process\_video`, only overrides seams; F1/F3 apply correctly.
4. **Control plane** ? all consumers correctly treat non-"success" report as retry/no-usable-result; cloud_run uses done-marker rc; restart-recovery now terminates fast. No regressions.

The one real defect I found is the **thread-ordering flakiness** in the two new ``test_partial_handoff_error_is_a_success_not_a_failure`` tests (trajectory + yolo). Let me do one final confirmation that the executor truly runs the two windows concurrently and that ``side_effect`` binds to call-arrival order, by re-reading the exact submission code path once more ? already confirmed at trajectory_hybrid lines 468-477 and yolo_hybrid lines 399-407. Both submit all windows to a ``max_workers=5`` pool, so 2 windows run on 2 threads concurrently, and ``MagicMock.side_effect`` list is consumed per-call in arrival order.

Now I'll report my findings.

44. user (2026-07-06T18:03:54.031Z)

Structured output provided successfully